



UNIVERSIDAD DE MURCIA

FACULTAD DE INFORMÁTICA

TRABAJO FIN DE MÁSTER

Análisis de la gestión de memoria GPU durante el entrenamiento con PyTorch y CUDA

Autor:

Juan García Alós

Tutor:

José Manuel García Carrasco

22 de junio de 2026

Índice

	Página
Lista de acrónimos	v
Resumen	1
Extended Abstract	2
1. Introducción	8
2. Background	10
2.1. Entrenamiento en GPU con PyTorch y CUDA	10
2.2. Gestión de memoria GPU en PyTorch	12
2.3. Modelos utilizados	14
2.4. Técnicas de optimización de memoria	15
2.5. Monitorización y benchmarks de entrenamiento	17
3. Estado del Arte	20
3.1. Gestión de memoria GPU en PyTorch y CUDA	20
3.2. Técnicas de reducción de memoria durante el entrenamiento	21
3.3. Atención eficiente y modelos Transformer	23
3.4. Entrenamiento eficiente a gran escala	23
3.5. Herramientas y guías prácticas de rendimiento	24
4. Objetivos y Metodología	25
4.1. Objetivos	25
4.2. Metodología	26
5. Diseño de la Solución	28
5.1. Visión general del sistema	28
5.2. Estructura del proyecto	29
5.3. Modelos implementados	30
5.4. Preparación de datos, entrenamiento y registro de métricas	31
5.5. Diagnóstico de memoria y configuraciones del allocator	32
5.6. Ejecución con SLURM y experimentos definidos	32

6. Análisis de Resultados	33
6.1. Entorno experimental y métricas	33
6.2. Resultados en BERT	34
6.3. Resultados en GPT-2	35
6.4. Resultados en ResNet	38
6.5. Comparación global de optimizaciones	40
6.6. Discusión general	42
7. Conclusiones y Vías Futuras	43

Índice de figuras

1.	Fases principales del entrenamiento y componentes de memoria implicados durante la ejecución.	11
2.	Visualización del estado interno del <i>CUDA caching allocator</i> mediante <i>Allocator State History</i> [29].	13
3.	Ejemplo de <i>Active Memory Timeline</i> en el visualizador de memoria de PyTorch [29].	18
4.	Arquitectura general del entorno desarrollado.	28
5.	Estructura general del proyecto.	29
6.	Comparación de memoria pico media en los experimentos con BERT. . . .	34
7.	Comparación de memoria pico media en los experimentos con GPT-2. . . .	36
8.	Evolución completa de la memoria en GPT-2 Large con SDPA, AMP y <i>gradient checkpointing</i>	37
9.	Detalle de los primeros eventos de memoria en GPT-2 Large con SDPA, AMP y <i>gradient checkpointing</i>	38
10.	Comparación de memoria pico media en los experimentos con ResNet. . . .	39
11.	Comparación del tiempo medio por batch en los experimentos con BERT. .	41
12.	Comparación del tiempo medio por batch en los experimentos con GPT-2. .	41
13.	Comparación del tiempo medio por batch en los experimentos con ResNet. .	42

Índice de tablas

1.	Resultados obtenidos en los experimentos con BERT.	34
2.	Resultados obtenidos en los experimentos con GPT-2.	35
3.	Resultados obtenidos en los experimentos con ResNet.	38

Lista de acrónimos

AMP Automatic Mixed Precision.

BERT Bidirectional Encoder Representations from Transformers.

CUDA Compute Unified Device Architecture.

GPT-2 Generative Pre-trained Transformer 2.

OOM Out Of Memory.

ResNet Residual Network.

SDPA Scaled Dot-Product Attention.

SLURM Simple Linux Utility for Resource Management.

Resumen

El crecimiento de los modelos de aprendizaje profundo ha incrementado de forma notable las necesidades de memoria durante el entrenamiento. Arquitecturas como los modelos Transformer o las redes convolucionales profundas requieren almacenar parámetros, gradientes, estados del optimizador, activaciones intermedias y buffers temporales en memoria GPU. Aunque las GPU actuales ofrecen una elevada capacidad de cómputo, la memoria disponible sigue siendo un recurso limitado y, en muchos casos, el principal factor que determina si una configuración de entrenamiento puede ejecutarse o termina en un error de falta de memoria.

En este contexto, la gestión eficiente de memoria GPU se convierte en un aspecto fundamental para entrenar modelos de forma escalable. PyTorch utiliza CUDA para ejecutar operaciones en GPU y gestiona la memoria mediante un mecanismo propio conocido como *CUDA caching allocator*. Este allocator reserva y reutiliza bloques de memoria para reducir el coste de las asignaciones, pero también hace necesario distinguir entre memoria realmente asignada por tensores activos y memoria reservada por PyTorch para futuras operaciones.

Este trabajo estudia la gestión de memoria GPU durante el entrenamiento de modelos de aprendizaje profundo utilizando PyTorch y CUDA. Para ello, se ha desarrollado un entorno experimental formado por tres modelos representativos: BERT, GPT-2 y ResNet. BERT se utiliza como modelo Transformer de tipo encoder para clasificación de texto, GPT-2 como modelo Transformer autoregresivo para modelado de lenguaje y ResNet como red convolucional profunda para clasificación de imágenes.

Para cada modelo se han implementado cargadores de datos, scripts de entrenamiento, generación de gráficas y registro de métricas. Durante cada ejecución se mide la memoria asignada, la memoria reservada, la memoria pico y el tiempo medio por batch. Además, se evalúan distintas técnicas de optimización: *Automatic Mixed Precision*, *gradient checkpointing*, AdamW de 8 bits, `scaled_dot_product_attention` y configuraciones del allocator como `max_split_size_mb` y `cudaMallocAsync`.

Los resultados muestran que la combinación de AMP y *gradient checkpointing* es la optimización más efectiva de forma general, reduciendo de manera significativa la memoria pico en BERT, GPT-2 y ResNet, y permitiendo ejecutar configuraciones que en modo base producían errores OOM. AdamW de 8 bits reduce la memoria asociada a los estados internos del optimizador, aunque su efecto es más limitado. SDPA mejora claramente el tiempo de ejecución en GPT-2, aunque apenas modifica la memoria pico cuando se combina con otras optimizaciones. Por último, las configuraciones del allocator modifican principalmente la memoria reservada, pero no resuelven por sí solas configuraciones cuyo consumo real supera la capacidad disponible de la GPU.

Extended Abstract

In recent years, deep learning models have grown significantly in size and complexity. This growth has produced important advances in natural language processing, computer vision and many other areas of artificial intelligence. However, it has also increased the computational cost of training these models. Modern neural networks require a large amount of GPU memory to store parameters, gradients, optimizer states, activations and temporary buffers. As a result, GPU memory has become one of the main limitations when training deep learning models, even when high-end hardware is available.

This problem is especially relevant because the available memory of a GPU is limited and cannot be increased as easily as storage or RAM. A model may have a reasonable execution time from a computational point of view, but still fail because the training process requires more memory than the GPU can provide. This situation usually appears as an out-of-memory error, also known as OOM. For this reason, efficient GPU memory management is essential for training large models, increasing batch size, using longer sequences or processing images with higher resolution.

PyTorch is one of the most widely used frameworks for deep learning research and development. It provides a flexible programming model, automatic differentiation and strong integration with CUDA, which allows tensors and operations to be executed on NVIDIA GPUs [25]. PyTorch does not request and release memory directly for every operation. Instead, it uses a mechanism called the CUDA caching allocator, which keeps memory blocks reserved in order to reuse them later [28, 7]. This strategy improves performance because it reduces the number of expensive memory allocation calls to CUDA. Nevertheless, it also means that the memory reported by external tools such as `nvidia-smi` may be higher than the memory currently used by active tensors.

Allocated memory is the memory occupied by active tensors that are still needed by the program. Reserved memory is the memory that PyTorch has requested from CUDA and keeps under the control of the caching allocator. This reserved memory may include active blocks and also free blocks that PyTorch keeps for future operations. Therefore, a training process can show a high amount of reserved memory even if the amount of allocated memory is lower. This distinction is important because some optimizations reduce the memory actually used by tensors, but do not always reduce the memory reserved by the allocator in the same proportion.

The main objective of this work is to study GPU memory management during the training of deep learning models using PyTorch and CUDA. The project does not propose a new memory optimization technique. Instead, it builds an experimental environment to compare existing techniques and analyse their behaviour in different types of models. The study focuses on practical optimizations available in PyTorch and commonly used in real training scenarios.

Three representative models were selected: BERT, GPT-2 and ResNet. BERT is used as a Transformer encoder model for text classification [8]. GPT-2 is used as an autoregressive Transformer model for language modelling [35]. ResNet is used as a deep convolutional neural network for image classification [13]. This selection makes it possible to compare different memory patterns. Transformer models are strongly affected by sequence length, attention mechanisms and optimizer states, while convolutional networks are especially affected by depth and input image resolution.

For each model, a complete training pipeline was implemented. This includes a data loader, a model file, a training script and a plotting script. BERT uses the GLUE/SST-2 dataset for binary sentiment classification [40]. GPT-2 uses WikiText-2, a common dataset for language modelling [21]. ResNet uses Tiny ImageNet, which allows the evaluation of convolutional models with different image resolutions [18]. The code has a common module with shared utilities for paths, memory measurement, logging, plotting and execution on the GACOP computing cluster through SLURM.

The experimental environment records several metrics during training. For each batch, memory is measured at different points: at the start of the batch, after the forward pass, after the backward pass and after the optimizer step. The main metrics are allocated memory, reserved memory, peak allocated memory and batch time. Batch time is measured using CUDA events, which provide more accurate timing for GPU operations.

Several optimization techniques were evaluated. The first one is Automatic Mixed Precision, or AMP. AMP combines operations in full precision and reduced precision, usually FP32 and FP16, in order to reduce memory usage and improve performance on modern GPUs [22, 27]. The second technique is gradient checkpointing. During training, the forward pass produces intermediate activations that are later needed during the backward pass. Gradient checkpointing reduces memory by storing fewer activations and recomputing part of the forward pass during backpropagation [3, 33]. This creates a trade-off between memory and computation time.

The third technique is the use of 8-bit optimizers. Optimizers such as Adam and AdamW store additional states for each model parameter [16, 20]. These states can take a large amount of memory in Transformer models. The 8-bit AdamW optimizer reduces this memory by quantizing the internal optimizer states [5, 1]. This technique was evaluated in BERT. The fourth technique is `scaled_dot_product_attention`, or SDPA. PyTorch provides this function as an interface for optimized attention implementations when they are available [32]. This is especially relevant in Transformer models, where attention can require a large amount of memory because the attention matrix grows quadratically with sequence length. Efficient attention implementations such as FlashAttention aim to reduce memory traffic and improve performance [4].

The experiments with BERT were designed to evaluate model scaling, sequence length

and optimizer memory. BERT-Base and BERT-Large were compared using a sequence length of 128 tokens. BERT-Base reached a peak memory of 4693.90 MB, while BERT-Large reached 12794.26 MB. This shows a strong memory increase when moving to the larger model, approximately 2.7 times higher. The average batch time also increased from 187.12 ms to 349.75 ms. This behaviour is expected because BERT-Large has more layers, a larger hidden dimension and more attention heads.

The combination of AMP and gradient checkpointing had a strong effect on BERT-Large. With a sequence length of 128 tokens, peak memory was reduced from 12794.26 MB to 7619.84 MB. This represents a reduction of approximately 40.44 %. Reserved memory was also reduced from 13084 MB to 7848 MB. In addition, the average batch time decreased from 349.75 ms to 259.76 ms. Although gradient checkpointing introduces re-computation, the use of AMP compensates for this cost in this case and improves the execution time.

BERT-Large with a sequence length of 512 tokens produced an OOM error in the base configuration. The recorded reserved memory values were close to the limit of the GPU. However, the optimized configuration using AMP and gradient checkpointing was able to execute successfully. In this case, the peak memory was 7629.78 MB and the reserved memory was 9528 MB. The average batch time increased to 629.88 ms, which shows that longer sequences remain computationally expensive even when the memory peak is controlled. This result is important because it shows that memory optimization can transform a configuration that cannot be executed into a viable training process.

The comparison between standard AdamW and 8-bit AdamW was also performed on BERT-Large with a sequence length of 128 tokens. Standard AdamW produced a peak memory of 12794.26 MB, while 8-bit AdamW reduced it to 10906.21 MB. This represents a reduction of approximately 14.76 %. Reserved memory was also reduced from 13084 MB to 11102 MB. However, the batch time remained almost unchanged, moving from 355.88 ms to 355.44 ms. This is coherent with the nature of this optimization, because 8-bit AdamW reduces the memory of optimizer states but does not reduce the activations produced during the forward and backward passes. When 8-bit AdamW was tested with a sequence length of 512 tokens, the execution still produced an OOM error. Therefore, this optimization is useful, but it is not enough when the main memory bottleneck comes from activations and sequence length.

The experiments with GPT-2 focused on model scaling, AMP, gradient checkpointing and SDPA. All GPT-2 experiments used a sequence length of 512 tokens. In the base configuration, GPT-2 Base reached a peak memory of 9507.14 MB, while GPT-2 Medium reached 20275.98 MB. The average batch time increased from 226.05 ms to 457.74 ms. GPT-2 Large could not complete the first batch in the base configuration and produced an OOM error. This shows that GPT-2 Large is not viable in this setup without memory

optimizations.

AMP and gradient checkpointing had a very strong impact on GPT-2 Medium. Peak memory was reduced from 20275.98 MB to 8514.44 MB, which represents a reduction of approximately 58.01 %. Reserved memory was reduced from 21178 MB to 10628 MB. The average batch time also decreased from 457.74 ms to 378.38 ms. In GPT-2 Large, the optimized configuration was able to run successfully with a peak memory of 18349.68 MB and a reserved memory of 20872 MB.

The use of SDPA in GPT-2 Large produced an interesting result. When SDPA was applied without AMP and gradient checkpointing, the model still produced an OOM error. Therefore, SDPA alone was not enough to solve the memory problem in this scenario. However, when SDPA was combined with AMP and gradient checkpointing, the model executed successfully. The peak memory was almost identical to the optimized version without SDPA: 18342.90 MB compared with 18349.68 MB. The difference in memory was very small. Nevertheless, the average batch time decreased from 639.57 ms to 438.43 ms, which represents an improvement of approximately 31.45 %.

This result indicates that SDPA is working, but its main benefit in this configuration is execution time rather than peak memory reduction. A possible explanation is that AMP and gradient checkpointing already reduce a large part of the activation memory. Moreover, the global memory peak may be dominated by other components of the model, such as parameters, gradients, optimizer states or feed-forward activations. Therefore, SDPA can improve the attention computation without changing the global memory peak in a significant way. This behaviour is consistent with practical benchmarks of Transformer training, where the memory benefit of SDPA depends on sequence length, batch size and the other optimizations being used [15].

The experiments with ResNet studied model depth, input resolution and allocator configurations. ResNet-50 and ResNet-101 were first compared with an input resolution of 224×224 . In the base configuration, ResNet-50 reached a peak memory of 5584.04 MB, while ResNet-101 reached 8247.79 MB. The average batch time increased from 169.82 ms to 202.07 ms. This result is expected because ResNet-101 has more layers and produces more intermediate activations.

AMP and gradient checkpointing reduced memory very strongly in ResNet. In ResNet-50, peak memory decreased from 5584.04 MB to 1844.88 MB, a reduction of approximately 66.96 %. In ResNet-101, peak memory decreased from 8247.79 MB to 2856.76 MB, a reduction of approximately 65.36 %. However, unlike BERT and GPT-2, the optimized configuration increased the average batch time slightly. In ResNet-50, the time increased from 169.82 ms to 177.71 ms, while in ResNet-101 it increased from 202.07 ms to 224.01 ms. This happens because the recomputation introduced by gradient checkpointing is not fully compensated by AMP in this convolutional model. Even so, the memory reduction

is very significant.

The input resolution had a strong effect on ResNet-101. With images of 512×512 , the base configuration produced an OOM error. This is because the spatial size of the activation maps increases considerably with image resolution. The optimized version using AMP and gradient checkpointing was able to run successfully, with a peak memory of 12580.04 MB and a reserved memory of 15734 MB. The average batch time increased to 689.88 ms.

Two allocator configurations were also tested in ResNet-101 with images of 512×512 . The first one was `max_split_size_mb=128`. This configuration did not prevent OOM in the non-optimized version. When combined with AMP and gradient checkpointing, the peak memory was almost identical to the normal optimized configuration, 12583.46 MB compared with 12580.04 MB, but reserved memory increased from 15734 MB to 21278 MB. Therefore, this allocator configuration did not improve memory behaviour in this experiment. The second configuration was `cudaMallocAsync`. It also failed in the non-optimized version, but when it was combined with AMP and gradient checkpointing, reserved memory decreased from 15734 MB to 14208 MB. Peak memory remained almost unchanged, with 12574.14 MB. This suggests that `cudaMallocAsync` can improve the internal management of reserved memory, but it does not reduce the real memory needed by the model.

During the development of this project, support for PyTorch memory snapshots was also implemented as a complementary diagnostic tool. These snapshots make it possible to visualize the internal state of the CUDA caching allocator, including active blocks, reserved segments, allocation and deallocation events, and the temporal evolution of memory usage [29, 38]. In this work, memory snapshots were not used as the main quantitative source of results, because the comparison between models and configurations required homogeneous numerical metrics. Instead, the final analysis was based on CSV logs and `torch.cuda.memory_summary()`, while snapshots were considered an additional visual mechanism to understand specific executions and illustrate how the allocator behaves internally.

The global results show that there is no single optimization that solves every memory problem. AMP and gradient checkpointing were the most effective techniques in this work, especially when activations were the main memory bottleneck. AdamW 8-bit reduced optimizer memory, but did not solve OOM when activations dominated the consumption. SDPA improved execution time in GPT-2, but did not reduce the global memory peak in the optimized configuration. Allocator configurations affected reserved memory, but did not reduce the actual memory required by the model.

As future work, several extensions are possible. First, it would be useful to run more experiments with different batch sizes and sequence lengths, especially in GPT-2, in order

to isolate the memory effect of SDPA. In particular, comparing SDPA with and without gradient checkpointing could provide a clearer view of its impact on attention activations. Secondly, additional techniques such as activation compression, CPU-GPU offloading or distributed training strategies like ZeRO could be evaluated [36, 2, 41]. These techniques would make it possible to study larger models, although they would also require a more complex infrastructure.

Another future direction is to repeat the experiments on different GPUs and with different CUDA and PyTorch versions. This would help determine whether the results are stable across hardware and software environments. It would also be useful to use memory snapshots in a wider range of controlled executions, especially with smaller numbers of batches, in order to analyse allocator behaviour visually without replacing the quantitative comparison based on CSV logs. Finally, the experimental framework could be extended with additional models, such as Vision Transformers or larger language models. This would make it possible to evaluate whether the conclusions obtained with BERT, GPT-2 and ResNet also apply to more recent architectures.

In conclusion, this work shows that GPU memory management must be analysed carefully when training deep learning models. It is not enough to observe the total memory reported by external tools, it is necessary to distinguish between allocated memory, reserved memory, peak memory and execution time. This distinction makes it possible to understand which part of the training process is affected by each optimization. The results confirm that AMP, gradient checkpointing, 8-bit optimizers, SDPA and allocator configurations can all be useful, but their effectiveness depends on the model, the input size and the memory pattern of the training process.

1. Introducción

Los modelos de aprendizaje profundo han experimentado un crecimiento muy rápido en los últimos años, arquitecturas como los modelos Transformer o las redes convolucionales profundas se utilizan comúnmente en tareas de procesamiento de lenguaje natural, visión por computador, generación de texto, clasificación de imágenes y muchas otras aplicaciones. Estos avances permiten obtener resultados cada vez mejores, pero también han incrementado de forma considerable los requisitos computacionales necesarios para entrenar estos modelos.

Uno de los recursos más limitantes durante el entrenamiento es la memoria GPU. Aunque las GPU modernas ofrecen una gran capacidad de cómputo, su memoria disponible sigue siendo insuficiente para muchos modelos con un tamaño relativamente grande, ya que durante el entrenamiento se almacenan muchos elementos: parámetros del modelo, gradientes, estados del optimizador, activaciones intermedias y buffers temporales. En ocasiones, una configuración puede no ser viable porque la GPU no dispone de memoria suficiente, independientemente de la potencia de cálculo de la que disponga.

En este contexto, la gestión eficiente de memoria GPU resulta ser un aspecto fundamental para el entrenamiento de modelos de aprendizaje profundo. PyTorch utiliza Compute Unified Device Architecture (CUDA) para ejecutar operaciones sobre la GPU y emplea un mecanismo propio de gestión de memoria conocido como *CUDA caching allocator*. Este mecanismo reserva y reutiliza bloques de memoria con el objetivo de reducir el coste de las asignaciones, pero es necesario distinguir entre memoria asignada por tensores activos y memoria reservada por el propio framework, ya que la memoria que reserva el allocator puede no corresponderse con la memoria real asignada.

En los últimos años han surgido distintas técnicas para reducir el consumo de memoria o mejorar el rendimiento durante el entrenamiento. Algunas de ellas, como Automatic Mixed Precision (AMP), reducen el tamaño de determinadas operaciones utilizando precisión reducida. Otras, como *gradient checkpointing*, disminuyen la memoria necesaria para almacenar activaciones intermedias a cambio de recomputar parte del *forward* durante el *backward*. También existen técnicas centradas en reducir los estados internos del optimizador, como AdamW de 8 bits, o en optimizar operaciones concretas, como Scaled Dot-Product Attention (SDPA) en modelos Transformer.

Este trabajo estudia el comportamiento de la memoria GPU durante el entrenamiento de modelos de aprendizaje profundo utilizando PyTorch y CUDA. Para ello, se ha desarrollado un entorno experimental con tres modelos representativos: Bidirectional Encoder Representations from Transformers (BERT), Generative Pre-trained Transformer 2 (GPT-2) y Residual Network (ResNet). BERT es un modelo Transformer de tipo encoder utilizado en tareas de clasificación de texto; GPT-2 representa un modelo Transformer autoregresivo para modelado de lenguaje; y ResNet se trata de una red convolucional

profunda para clasificación de imágenes. Estos modelos han sido seleccionados por su relevancia dentro de la literatura científica y por su uso frecuente como arquitecturas de referencia en investigación, benchmarks y estudios de rendimiento.

El trabajo se ha centrado en implementar una infraestructura común para ejecutar experimentos, registrar métricas y generar gráficas comparativas. Para cada modelo se han desarrollado cargadores de datos, scripts de entrenamiento y scripts de visualización de resultados. Durante cada ejecución se registran métricas como la memoria asignada, la memoria reservada, el pico máximo de memoria y el tiempo medio por batch. Gracias a estas métricas se puede analizar cómo se comportan los modelos internamente a lo largo del entrenamiento.

Los resultados obtenidos muestran que la combinación de AMP y *gradient checkpointing* es la optimización más efectiva de forma general. Esta combinación reduce de manera significativa la memoria pico en los tres modelos analizados, y permite ejecutar configuraciones que en modo base terminan en errores de falta de memoria. AdamW de 8 bits reduce la memoria asociada a los estados internos del optimizador, aunque su efecto es más limitado. Por otro lado, SDPA mejora de forma clara el tiempo de ejecución en GPT-2, aunque su impacto sobre la memoria pico es reducido cuando se combina con otras optimizaciones. Las configuraciones del allocator modifican principalmente la memoria reservada, pero no reducen la memoria asignada necesaria para entrenar el modelo.

La estructura del documento se organiza de la siguiente manera: En la Sección 2 se presentan los conceptos necesarios para comprender la gestión de memoria GPU en PyTorch y CUDA. En la Sección 3 se revisan los trabajos relacionados con técnicas de optimización, monitorización de memoria y entrenamiento eficiente. La Sección 4 explica los objetivos del trabajo y la metodología seguida. El diseño de la solución, incluyendo la estructura del proyecto, los modelos utilizados, el sistema de entrenamiento y el registro de métricas, se detalla en la Sección 5. En la Sección 6 se muestran los resultados obtenidos, acompañados de un análisis por modelo y una comparación global de las optimizaciones. Por último, en la Sección 7 se recogen las conclusiones del trabajo, así como posibles líneas futuras de investigación.

2. Background

En esta sección se presentan los conceptos necesarios para comprender el trabajo realizado. En primer lugar, se introduce el entrenamiento de modelos de aprendizaje profundo en GPU y el papel de PyTorch y CUDA en este proceso. A continuación, se explica cómo PyTorch gestiona la memoria GPU, diferenciando entre memoria asignada, memoria reservada y memoria pico. Después se describen los modelos utilizados en el trabajo, junto con las principales técnicas de optimización evaluadas: AMP, *gradient checkpointing*, optimizadores de 8 bits y atención escalada optimizada.

2.1. Entrenamiento en GPU con PyTorch y CUDA

El entrenamiento de modelos de aprendizaje profundo consiste en ajustar los parámetros de una red neuronal a partir de un conjunto de datos. Para ello, se realiza un proceso iterativo en el que el modelo recibe un batch de entrada, calcula una salida, compara dicha salida con el valor esperado mediante una función de pérdida y actualiza sus parámetros mediante un algoritmo de optimización. Este proceso se repite durante múltiples batches y épocas, hasta que el modelo alcanza un comportamiento adecuado para la tarea objetivo.

Desde el punto de vista computacional, el entrenamiento se divide en tres fases principales: *forward*, *backward* y actualización del optimizador. Durante el *forward*, los datos atraviesan las capas del modelo y se generan las predicciones. Durante el *backward*, se calculan los gradientes de la función de pérdida respecto a los parámetros del modelo. Finalmente, el optimizador utiliza estos gradientes para actualizar los pesos. Estas operaciones implican una gran cantidad de productos matriciales, convoluciones y operaciones vectorizadas, por lo que las GPU son la mejor herramienta para acelerar el entrenamiento.

La GPU incorpora una memoria propia, conocida como VRAM, donde deben almacenarse los tensores necesarios para el entrenamiento. Durante una ejecución típica se almacenan los parámetros del modelo, los gradientes, los estados internos del optimizador, las entradas, las salidas intermedias de las capas y diferentes buffers temporales. Por ello, aunque una GPU tenga una gran capacidad de cálculo, el tamaño de su memoria puede convertirse en un cuello de botella a la hora de entrenar modelos grandes o utilizar batches grandes.

El consumo de memoria durante el entrenamiento no es constante, normalmente aumenta durante el *forward*, ya que se almacenan activaciones intermedias que posteriormente serán necesarias para calcular los gradientes. Durante el *backward*, parte de estas activaciones se utilizan y pueden liberarse, pero también se generan gradientes y otros tensores temporales. En la fase de actualización del optimizador pueden aparecer nuevos consumos, especialmente cuando se utilizan optimizadores como Adam o AdamW, que almacenan información adicional para cada parámetro. Por este motivo, medir únicamente la memoria

al inicio o al final de una ejecución no es suficiente para comprender el comportamiento real del entrenamiento.

En la Figura 1 se muestra de forma visual cómo se organiza el entrenamiento y qué componentes contribuyen al consumo de memoria durante la ejecución.

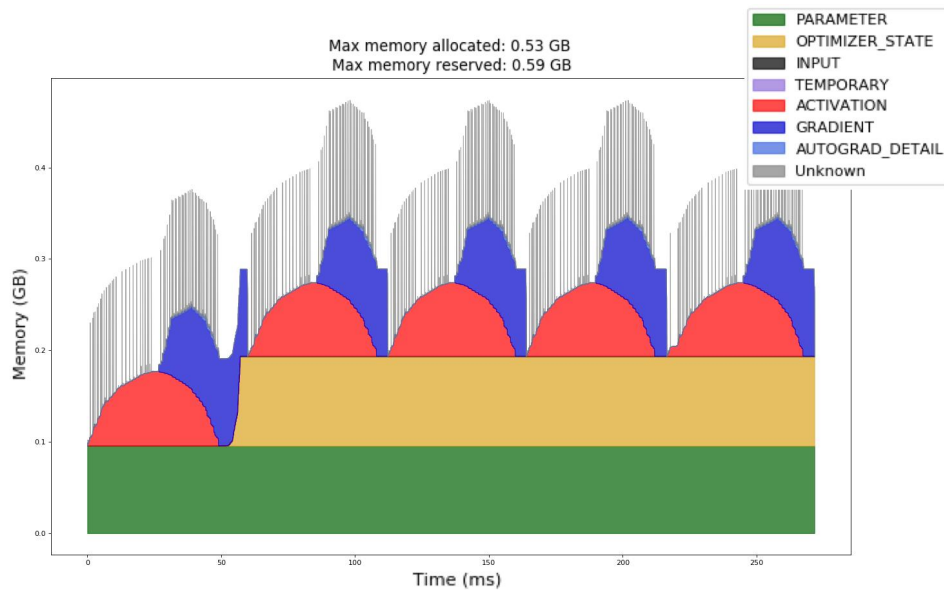


Figura 1: Fases principales del entrenamiento y componentes de memoria implicados durante la ejecución.

PyTorch es uno de los frameworks más utilizados para el desarrollo y entrenamiento de modelos de aprendizaje profundo. Su popularidad se debe, entre otros motivos, a su integración con Python, su modelo de ejecución imperativo y su sistema de diferenciación automática, que permite calcular gradientes de forma dinámica durante el entrenamiento [25]. Esta forma de trabajo facilita la experimentación, ya que el código del modelo se ejecuta de manera similar a un programa de Python convencional.

CUDA, por su parte, es la plataforma de NVIDIA que permite ejecutar cómputo general en GPU. PyTorch utiliza CUDA para mover tensores a la GPU y ejecutar operaciones sobre ellos de forma acelerada. En la práctica, cuando un tensor se mueve a un dispositivo CUDA, sus datos pasan a almacenarse en memoria GPU y las operaciones posteriores pueden ejecutarse sobre ese dispositivo. Este modelo permite aprovechar la capacidad de paralelismo de la GPU, pero también obliga a gestionar correctamente la memoria disponible.

PyTorch también debe solicitar memoria a CUDA, reutilizarla cuando sea posible y

liberarla cuando ya no sea necesaria. Para evitar que cada operación tenga que pedir y devolver memoria directamente al sistema CUDA, PyTorch utiliza un mecanismo propio de gestión conocido como *CUDA caching allocator*. Este allocator es una pieza central del trabajo, y explica por qué la memoria que aparece como utilizada en herramientas externas no siempre coincide con la memoria realmente ocupada por tensores activos [28, 7].

2.2. Gestión de memoria GPU en PyTorch

La gestión de memoria en PyTorch puede ser algo confusa si solo se observa la memoria mostrada por herramientas como `nvidia-smi`. Esto ocurre porque PyTorch no siempre libera toda la memoria del sistema cuando los tensores dejan de utilizarse. Puede conservar bloques de memoria previamente solicitados para reutilizarlos en futuras operaciones. Esta estrategia reduce el coste de nuevas asignaciones, pero provoca que la memoria reservada por PyTorch pueda ser mayor que la memoria realmente ocupada por tensores activos.

2.2.1. Memoria asignada y memoria reservada

Una distinción fundamental en PyTorch es la diferencia entre memoria asignada y memoria reservada. La memoria asignada representa la memoria ocupada por tensores activos, es decir, aquellos tensores que siguen siendo necesarios en el programa. Esta métrica puede obtenerse mediante funciones como `torch.cuda.memory_allocated()`.

La memoria reservada describe la memoria que PyTorch ha solicitado al sistema CUDA y mantiene bajo control del allocator. Esta memoria puede incluir tanto bloques activos como bloques libres que PyTorch conserva para reutilizarlos posteriormente. Esta métrica puede consultarse mediante `torch.cuda.memory_reserved()` [28].

En algunos casos, una optimización puede reducir la memoria asignada, pero no modificar de forma inmediata la memoria reservada. En otros casos, una configuración del allocator puede cambiar la forma en que PyTorch reserva bloques, sin reducir necesariamente el consumo real del modelo.

2.2.2. CUDA caching allocator

El *CUDA caching allocator* de PyTorch es el componente encargado de gestionar la memoria GPU reservada por el framework. Su objetivo principal es evitar llamadas frecuentes a las funciones de asignación y liberación de CUDA, ya que estas operaciones pueden introducir sincronizaciones y afectar al rendimiento. Para ello, PyTorch conserva bloques de memoria ya solicitados y los reutiliza cuando aparecen nuevas peticiones compatibles [7].

Si un entrenamiento utiliza tamaños de tensor similares entre batches, el allocator puede reutilizar bloques de memoria y reducir el coste de asignación. Sin embargo, también puede

generar situaciones donde la memoria reservada sea elevada aunque la memoria asignada sea menor. Esto no implica necesariamente una fuga de memoria, sino que puede ser consecuencia del funcionamiento normal del allocator.

Otro aspecto relevante es la fragmentación. La memoria reservada puede quedar dividida en bloques que no siempre se ajustan de forma eficiente a nuevas peticiones. En estos casos, aunque exista memoria libre dentro del allocator, puede no estar disponible en bloques suficientemente grandes para una nueva asignación. PyTorch permite modificar ciertos aspectos del comportamiento del allocator mediante variables de entorno, como `PYTORCH_CUDA_ALLOC_CONF`. Entre las opciones disponibles se encuentra `max_split_size_mb`, que limita el tamaño máximo de los bloques que pueden dividirse, y `backend:cudaMallocAsync`, que permite utilizar un backend alternativo basado en el allocator asíncrono de CUDA [28, 24, 17].

En la Figura 2 se observa una visualización del estado interno del allocator, donde pueden identificarse segmentos reservados y bloques de memoria activos o libres.

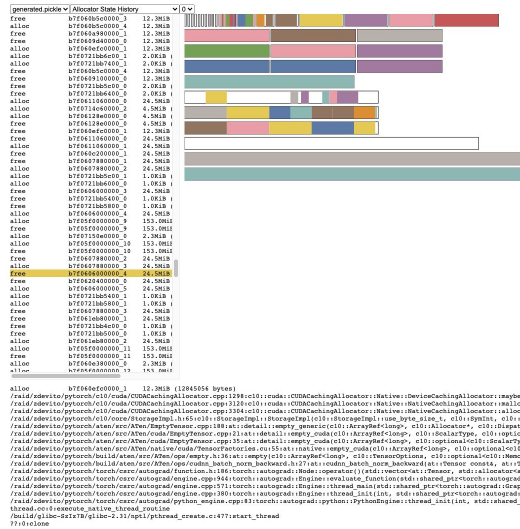


Figura 2: Visualización del estado interno del *CUDA caching allocator* mediante *Allocator State History* [29].

2.2.3. Errores de falta de memoria

Un error de tipo Out Of Memory (OOM) se produce cuando PyTorch no puede satisfacer una nueva petición de memoria en la GPU. Este error puede deberse a que el modelo, el batch o la entrada son demasiado grandes para la memoria disponible. También puede estar relacionado con la fragmentación de memoria o con una acumulación de tensores que siguen referenciados en el programa.

En los mensajes de error de PyTorch suelen aparecer varias métricas: memoria total de la GPU, memoria libre, memoria asignada por PyTorch, memoria reservada pero no asignada y, en algunos casos, recomendaciones relacionadas con el allocator. Esta información es útil para interpretar si el problema parece estar causado por un consumo real excesivo o por una posible fragmentación.

2.3. Modelos utilizados

Para analizar distintos patrones de uso de memoria se han seleccionado tres modelos: BERT, GPT-2 y ResNet. Los dos primeros son de tipo Transformer, mientras que ResNet es una red convolucional profunda. Esta selección permite comparar modelos con características internas distintas, tanto en el tipo de operación dominante como en la forma en que crece el consumo de memoria.

2.3.1. Arquitecturas Transformer

Las arquitecturas Transformer fueron introducidas como una alternativa a los modelos recurrentes para tareas de procesamiento de secuencias [39]. Su componente principal es el mecanismo de atención, que permite que cada posición de una secuencia tenga en cuenta información procedente de otras posiciones. Este enfoque es muy importante en procesamiento de lenguaje natural, donde ha permitido construir modelos capaces de capturar dependencias complejas entre palabras o tokens.

El mecanismo de atención escalada se basa en tres matrices: consultas, claves y valores. Las consultas se comparan con las claves para obtener una matriz de puntuaciones, que después se normaliza y se utiliza para ponderar los valores. En una implementación estándar, esta matriz de atención tiene un tamaño proporcional a la longitud de secuencia al cuadrado. Por ello, el consumo de memoria asociado a la atención crece rápidamente cuando aumenta la longitud de entrada.

Modelos como BERT y GPT-2 utilizan múltiples capas de atención multi-cabeza. El número de capas, el número de cabezas, la dimensión oculta y la longitud de secuencia influyen directamente en el coste computacional y en el uso de memoria.

2.3.2. BERT

BERT es un modelo Transformer encoder diseñado para obtener representaciones bidireccionales del texto [8]. A diferencia de modelos autoregresivos, BERT puede atender simultáneamente a tokens situados a la izquierda y a la derecha de una posición, lo que resulta útil para tareas de comprensión del lenguaje. En este trabajo se utiliza como representante de modelos encoder aplicados a clasificación de texto.

Desde el punto de vista de memoria, su consumo está muy condicionado por la longitud de secuencia y por el tamaño del modelo. Las dos implementaciones de este trabajo, BERT-Base y BERT-Large, difieren en número de capas, dimensión oculta y número de cabezas de atención. Al aumentar estos valores, crecen tanto el número de parámetros como las activaciones intermedias generadas durante el entrenamiento.

2.3.3. GPT-2

GPT-2 es un modelo Transformer autoregresivo diseñado para modelado de lenguaje [35]. Su objetivo es predecir el siguiente token de una secuencia a partir de los tokens anteriores. Para ello utiliza atención causal, de forma que cada posición solo puede atender a posiciones previas. Esta restricción permite generar texto de izquierda a derecha, pero también mantiene un coste de atención elevado cuando la longitud de secuencia aumenta.

En este trabajo, se implementan tres variantes del modelo, Base, Medium y Large, de forma que se pueda estudiar el escalado de memoria con respecto al tamaño de la red. También se estudia el efecto de realizar el cálculo de la atención de forma manual mediante `scaled_dot_product_attention`.

2.3.4. ResNet

ResNet es una arquitectura convolucional profunda basada en conexiones residuales [13]. Estas conexiones permiten sumar la entrada de un bloque a su salida, facilitando el entrenamiento de redes más profundas sin que se produzca la degradación de rendimiento típica de las redes convolucionales. En este trabajo se utilizan ResNet-50 y ResNet-101, que son variantes basadas en bloques *bottleneck*.

A diferencia de los modelos Transformer, ResNet no utiliza mecanismos de atención. Su consumo de memoria está más relacionado con el número de capas convolucionales, el tamaño de los mapas de activación y la resolución de entrada.

2.4. Técnicas de optimización de memoria

Existen distintas técnicas para reducir el consumo de memoria durante el entrenamiento. Algunas actúan sobre el tipo de dato utilizado en las operaciones, otras reducen las activaciones almacenadas y otras modifican la representación de los estados internos del optimizador. En este trabajo se evalúan varias de estas técnicas con el objetivo de analizar su impacto en diferentes modelos.

2.4.1. AMP

AMP es una técnica que combina operaciones en precisión completa y precisión reducida durante el entrenamiento. En lugar de ejecutar todo el modelo en FP32, algunas

operaciones pueden realizarse en FP16, reduciendo el uso de memoria y aprovechando unidades especializadas de las GPU modernas. Esta técnica puede mejorar el rendimiento y disminuir el consumo de memoria, aunque requiere mecanismos adicionales para mantener la estabilidad numérica [22].

En PyTorch, AMP se utiliza habitualmente mediante `autocast` y `GradScaler`. `autocast` selecciona automáticamente qué operaciones pueden ejecutarse en precisión reducida, mientras que `GradScaler` ayuda a evitar problemas de subdesbordamiento en los gradientes cuando se utiliza FP16 [27, 26].

AMP no reduce todos los componentes de memoria por igual. Los parámetros, gradientes, estados del optimizador y buffers temporales pueden verse afectados de forma distinta dependiendo del modelo y de la operación. Por este motivo, es importante analizar su efecto empíricamente en cada arquitectura.

2.4.2. Gradient checkpointing

Durante el entrenamiento, las activaciones intermedias generadas en el *forward* suelen almacenarse para ser utilizadas posteriormente en el *backward*. En modelos profundos, estas activaciones pueden ocupar una parte muy importante de la memoria GPU. *Gradient checkpointing* reduce este consumo evitando almacenar todas las activaciones y recomputando parte de ellas durante el cálculo de gradientes [3].

Esta técnica introduce un intercambio entre memoria y tiempo. Al almacenar menos activaciones, se reduce la memoria necesaria durante el entrenamiento. Sin embargo, al recomputar parte del *forward*, aumenta el coste computacional. Este intercambio permite entrenar modelos grandes que de otro modo no cabrían en memoria o aumentar el batch size o la longitud de entrada.

La idea de recomputar activaciones para reducir memoria aparece también en otros trabajos relacionados con entrenamiento eficiente, como la retropropagación eficiente en memoria para modelos recurrentes [11]. En PyTorch, esta técnica está disponible mediante `torch.utils.checkpoint`, que permite aplicar checkpointing a módulos concretos del modelo [33, 34].

2.4.3. Optimizadores de 8 bits

Los optimizadores adaptativos, como Adam y AdamW, mantienen estados internos adicionales para cada parámetro. En Adam, se almacenan estimaciones de primer y segundo momento de los gradientes [16]. AdamW modifica el tratamiento del *weight decay*, desacoplándolo de la actualización adaptativa [20]. Estos optimizadores son muy utilizados en modelos Transformer, pero sus estados pueden representar una parte relevante del consumo total de memoria.

Los optimizadores de 8 bits buscan reducir este coste almacenando los estados internos del optimizador con una representación cuantizada. El trabajo de Dettmers et al. propone optimizadores de 8 bits mediante cuantización por bloques, reduciendo la memoria necesaria para los estados sin afectar de forma significativa al comportamiento del entrenamiento en muchos escenarios [5]. En este trabajo se utiliza AdamW de 8 bits mediante la librería `bitsandbytes` para evaluar su impacto en BERT [1].

2.4.4. SDPA

SDPA es una de las operaciones centrales en los modelos Transformer. Una implementación directa de la atención puede crear tensores intermedios grandes, especialmente la matriz de puntuaciones de atención y la matriz resultante tras aplicar *softmax*. Como estas matrices crecen cuadráticamente con la longitud de secuencia, su impacto en memoria resulta elevado en secuencias largas.

PyTorch proporciona la función `torch.nn.functional.scaled_dot_product_attention`, que actúa como una interfaz para ejecutar atención escalada utilizando distintas implementaciones disponibles según el hardware, el tipo de dato y la forma de los tensores [32]. Entre estas implementaciones pueden encontrarse kernels optimizados como FlashAttention o variantes eficientes en memoria. FlashAttention propone una atención exacta consciente de la jerarquía de memoria GPU, reduciendo accesos a memoria de alto coste y evitando materializar algunos tensores intermedios grandes [4].

En este trabajo se compara una implementación manual de atención en GPT-2 con una versión basada en SDPA. El objetivo es analizar si el uso de una implementación optimizada se traduce en menor tiempo de entrenamiento, menor memoria pico o ambas cosas. SDPA no garantiza siempre un ahorro de memoria elevado, su beneficio depende de si la atención domina o no el pico de memoria del entrenamiento, de la longitud de secuencia, del batch size y de otras optimizaciones activadas, como AMP o *gradient checkpointing*.

2.5. Monitorización y benchmarks de entrenamiento

Para estudiar el comportamiento de memoria es necesario registrar métricas internas durante la ejecución y analizar cómo evolucionan. PyTorch ofrece distintas herramientas para consultar el estado de la memoria CUDA, como `memory_allocated`, `memory_reserved`, `max_memory_allocated` y `memory_summary` [28].

`memory_summary` genera un resumen textual con información sobre memoria asignada, memoria activa, memoria reservada, memoria no liberable, número de asignaciones, segmentos reservados y otros indicadores. Esta información resulta útil para obtener una visión general del estado del allocator y comprobar si existe una diferencia significativa

entre la memoria realmente asignada por tensores activos y la memoria reservada por PyTorch.

Además, PyTorch dispone de herramientas de *memory snapshot* para visualizar asignaciones y liberaciones de memoria a lo largo del tiempo [29, 38]. Estas herramientas permiten obtener una representación más detallada del comportamiento interno del allocator, mostrando bloques activos, bloques reservados, eventos de asignación y liberación, y trazas asociadas a las operaciones que generan consumo de memoria.

En la Figura 3 se muestra una vista temporal de las asignaciones activas, lo que permite observar la evolución de la memoria durante una ejecución concreta.

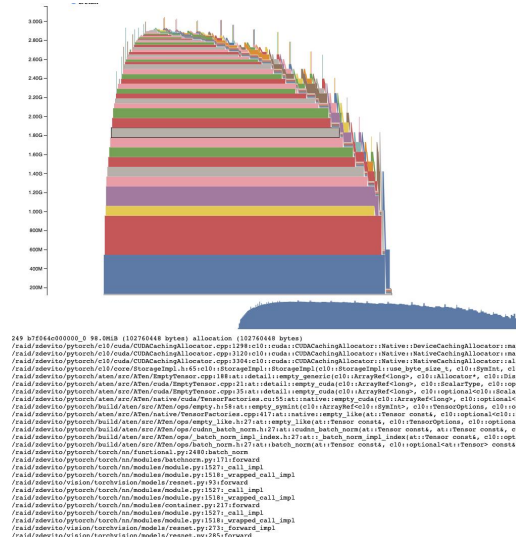


Figura 3: Ejemplo de *Active Memory Timeline* en el visualizador de memoria de PyTorch [29].

La monitorización de memoria también ha sido tratada en publicaciones técnicas orientadas al uso práctico de PyTorch, donde se destaca la importancia de distinguir entre memoria asignada, reservada y pico de memoria para interpretar correctamente el comportamiento de los entrenamientos [9, 14, 10].

El análisis de rendimiento en modelos de aprendizaje profundo requiere comparar configuraciones bajo condiciones controladas. Para ello, es necesario fijar aspectos como el modelo, el tamaño de entrada, el batch size, el número de batches y las técnicas activadas. En PyTorch existen trabajos y herramientas orientadas al benchmarking de modelos y APIs del framework, como TorchBench, que propone una colección de modelos para evaluar rendimiento y cobertura de la superficie de APIs de PyTorch [12]. Las métricas

principales utilizadas en este trabajo son la memoria asignada, la memoria reservada, el pico máximo de memoria y el tiempo medio por batch.

3. Estado del Arte

Esta sección revisa los trabajos más relevantes relacionados con la optimización y monitorización de memoria GPU durante el entrenamiento de modelos de aprendizaje profundo. En primer lugar, se presentan trabajos y publicaciones centradas en la gestión de memoria en PyTorch y CUDA. A continuación, se revisan técnicas de reducción de memoria durante el entrenamiento, como la recomputación de activaciones, la precisión mixta, la cuantización de estados del optimizador y las implementaciones eficientes de atención. Por último, se analizan trabajos relacionados con benchmarking y herramientas prácticas para evaluar el rendimiento de modelos en PyTorch.

3.1. Gestión de memoria GPU en PyTorch y CUDA

La gestión de memoria GPU se ha convertido en un aspecto fundamental en el entrenamiento de modelos de aprendizaje profundo. El crecimiento de los modelos, el aumento de la longitud de las secuencias y el uso de batches grandes hacen que la memoria disponible en GPU sea uno de los factores más limitantes. PyTorch incorpora un sistema propio de gestión de memoria, conocido como *CUDA caching allocator*, cuyo objetivo es reducir el coste de las asignaciones y liberaciones de memoria reutilizando bloques previamente reservados [28].

Uno de los trabajos técnicos más relevantes para comprender este comportamiento es la publicación de DeVito [7], donde se explica el funcionamiento interno del *CUDA caching allocator* de PyTorch. En este trabajo se describe cómo PyTorch intenta alcanzar un estado estable de memoria, reutilizando bloques entre iteraciones para reducir las llamadas a CUDA. También se explica por qué la memoria reservada puede permanecer alta aunque los tensores activos ocupen menos memoria. Es importante tener en cuenta este comportamiento a la hora de interpretar correctamente como se comportan la memoria asignada y la memoria reservada, ya que una memoria reservada elevada no implica necesariamente una fuga de memoria.

Dhanjal [9] aborda la gestión de memoria GPU en PyTorch y CUDA desde un punto de vista práctico. En esta publicación se revisan conceptos como la memoria asignada, la memoria reservada, el uso de `nvidia-smi` y la importancia de liberar referencias innecesarias a tensores.

También de forma práctica, Amit [14] presenta técnicas de perfilado de memoria en PyTorch, destacando la importancia de medir el consumo durante la ejecución y no únicamente al finalizar el entrenamiento. Estas publicaciones muestran que muchos problemas de memoria no se detectan observando únicamente el consumo global de GPU. Requieren analizar cómo evoluciona la memoria durante el *forward*, el *backward* y la actualización del optimizador.

Además de estas publicaciones, PyTorch proporciona herramientas oficiales para analizar la memoria CUDA, como `memory_allocated`, `memory_reserved`, `memory_summary` y los *memory snapshots* [28, 29]. Los *memory snapshots* permiten capturar el estado interno del allocator y visualizar asignaciones y liberaciones de memoria a lo largo del tiempo. Shi y DeVito [38] presentan el uso de estas herramientas para visualizar todas las asignaciones de memoria durante una ejecución. Esto permite detectar problemas de fragmentación, bloques no liberables o picos de memoria difíciles de interpretar únicamente mediante métricas agregadas.

Estas herramientas son especialmente útiles como complemento a las métricas numéricas, ya que permiten observar de forma visual cómo se organizan las asignaciones dentro del allocator y en qué momentos aparecen determinados bloques de memoria. En este trabajo se implementó soporte para *memory snapshots* como una funcionalidad adicional de diagnóstico. No obstante, el análisis principal se basa en los registros CSV y en `memory_summary`, ya que estas métricas permiten realizar una comparación homogénea entre modelos, configuraciones y técnicas de optimización.

3.2. Técnicas de reducción de memoria durante el entrenamiento

La literatura ha propuesto distintas estrategias para reducir el consumo de memoria durante el entrenamiento de redes neuronales profundas. Estas técnicas actúan sobre diferentes componentes del entrenamiento: activaciones intermedias, precisión numérica, estados del optimizador, comunicación entre dispositivos o almacenamiento temporal en CPU. En este trabajo se han implementado algunas de estas técnicas y se han analizado otras como parte del estado del arte.

3.2.1. Recomputación de activaciones

Una de las técnicas más utilizadas para reducir memoria es la recomputación de activaciones, también conocida como *gradient checkpointing*. Durante el entrenamiento, las activaciones generadas en el *forward* deben conservarse para calcular gradientes durante el *backward*. En modelos profundos, estas activaciones pueden ocupar una cantidad considerable de VRAM. La idea de *gradient checkpointing* consiste en almacenar solo algunas activaciones seleccionadas y recomputar el resto cuando sean necesarias.

El trabajo de Chen et al. [3] es una de las referencias principales. Los autores proponen entrenar redes profundas con coste de memoria sublineal, reduciendo el número de activaciones almacenadas a cambio de realizar cómputo adicional durante el *backward*. Este planteamiento produce un intercambio entre memoria y tiempo: se reduce el consumo de memoria, pero aumenta el coste computacional por la necesidad de repetir parte del *forward*.

Gruslys et al. [11] estudian una idea relacionada en el contexto de modelos recurrentes, proponiendo estrategias de retropropagación eficientes en memoria. Aunque su trabajo se centra en otro tipo de arquitectura, comparte el mismo principio general, reducir almacenamiento intermedio a cambio de recomputar operaciones.

En PyTorch, esta técnica está disponible mediante `torch.utils.checkpoint` [33]. Además, publicaciones recientes de la PyTorch Foundation [34] revisan nuevas técnicas de checkpointing y muestran que sigue siendo una línea activa de optimización.

3.2.2. Precisión mixta

Otra técnica ampliamente utilizada es el entrenamiento en precisión mixta. El trabajo de Micikevicius et al. [22] introduce *Mixed Precision Training*, una técnica que combina operaciones en FP32 y FP16 para reducir memoria y acelerar el entrenamiento en GPU. La precisión reducida permite almacenar ciertos tensores con menor tamaño y aprovechar unidades especializadas de las GPU modernas, como los Tensor Cores.

El uso de precisión mixta requiere tener en cuenta la estabilidad numérica. Algunas operaciones pueden ejecutarse de forma segura en FP16, mientras que otras deben mantenerse en FP32. Además, al trabajar con gradientes pequeños puede producirse subdesbordamiento, por lo que se utilizan técnicas como el escalado de pérdida. En PyTorch, este proceso se automatiza mediante `autocast` y `GradScaler`, integrados dentro del paquete `torch.amp` [27, 26].

AMP se ha convertido en una técnica práctica habitual porque puede mejorar simultáneamente el tiempo de entrenamiento y el consumo de memoria. A diferencia de *gradient checkpointing*, que suele aumentar el tiempo de cómputo, AMP puede reducir el tiempo si las operaciones se benefician de hardware especializado. Sin embargo, su impacto depende del modelo, del tipo de operaciones y del hardware utilizado.

3.2.3. Cuantización de estados del optimizador

Además de parámetros, gradientes y activaciones, los optimizadores pueden ocupar una parte importante de la memoria. Esto es especialmente relevante en optimizadores adaptativos como Adam y AdamW, que almacenan estados adicionales para cada parámetro. Kingma y Ba [16] propusieron Adam como un optimizador basado en estimaciones de primer y segundo momento de los gradientes. Posteriormente, Loshchilov y Hutter [20] propusieron AdamW, desacoplando el *weight decay* de la actualización adaptativa.

El trabajo de Dettmers et al. [5] propone optimizadores de 8 bits mediante cuantización por bloques. La idea principal consiste en almacenar los estados internos del optimizador con menor precisión, reduciendo el consumo de memoria sin perjudicar significativamente al entrenamiento. Esta técnica es más interesante en modelos Transformer, donde el núme-

ro de parámetros es elevado y los estados del optimizador pueden representar una fracción considerable de la memoria total. La librería `bitsandbytes` proporciona implementaciones prácticas de optimizadores de 8 bits, incluyendo AdamW8bit [1].

3.3. Atención eficiente y modelos Transformer

Los modelos Transformer han contribuido de forma significativa al aumento de los requisitos de memoria en aprendizaje profundo, especialmente en tareas con secuencias largas. Desde la introducción de la arquitectura Transformer por Vaswani et al. [39], este tipo de modelos se ha convertido en la base de arquitecturas como BERT [8] y GPT-2 [35]. Su componente principal, la atención multi-cabeza, permite capturar dependencias entre posiciones de una secuencia, pero introduce un coste de memoria que crece cuadráticamente con la longitud de entrada. Este crecimiento cuadrático se debe a la matriz de atención, cuyo tamaño depende de la longitud de secuencia al cuadrado. En secuencias cortas, el tamaño en memoria que ocupa esta matriz puede no ser relevante, ya que también influyen otros parámetros como gradientes, activaciones de capas feed-forward y estados del optimizador. Sin embargo, al aumentar la longitud de secuencia, la atención puede convertirse en el principal cuello de botella para la memoria.

FlashAttention, propuesto por Dao et al. [4], aborda este problema teniendo en cuenta la jerarquía de memoria GPU. En lugar de materializar de forma directa todos los tensores intermedios de atención, reorganiza el cálculo para reducir accesos a memoria de alto coste y mejorar la eficiencia. Esta técnica mantiene una atención exacta, pero reduce el tráfico de memoria y mejora el rendimiento. PyTorch incorpora `scaled_dot_product_attention` como una interfaz capaz de seleccionar distintas implementaciones disponibles para la atención escalada, incluyendo implementaciones optimizadas cuando se cumplen las condiciones necesarias [32, 30]. Esto permite que ciertos modelos puedan beneficiarse de kernels más eficientes sin implementar manualmente FlashAttention.

Es importante destacar que el beneficio de SDPA no siempre se refleja de la misma forma en memoria y tiempo. En algunos escenarios puede reducir notablemente el tiempo de ejecución, pero apenas modificar la memoria pico si esta se encuentra dominada por otros componentes del entrenamiento. Este comportamiento ha sido documentado también en guías prácticas de entrenamiento eficiente en GPU, donde se observa que técnicas como SDPA, AMP y checkpointing pueden solaparse parcialmente en su efecto sobre las activaciones [15].

3.4. Entrenamiento eficiente a gran escala

Además de las líneas analizadas en este trabajo, existen otras líneas de investigación orientadas a entrenar modelos de gran escala en entornos distribuidos o con estrategias más complejas de gestión de memoria. Aunque estas técnicas no se implementan en este

TFM, son relevantes para situar el trabajo dentro del contexto general de entrenamiento eficiente.

Una de las propuestas más influyentes es ZeRO, presentada por Rajbhandari et al. [36]. ZeRO busca reducir la redundancia de memoria en entrenamiento distribuido dividiendo parámetros, gradientes y estados del optimizador entre varios dispositivos. Esta estrategia permite entrenar modelos de muchos más parámetros al evitar que cada GPU tenga que almacenar una copia completa de todos los estados.

Otra línea es el *offloading*, donde parte de la información se mueve entre GPU y CPU para reducir la presión sobre la memoria de la GPU. Rhu et al. [37] presentan vDNN, una propuesta para virtualizar redes neuronales profundas y permitir entrenamientos más grandes mediante transferencia de datos entre CPU y GPU. Este tipo de técnicas puede ampliar el tamaño de modelos entrenables a cambio de introducir costes asociados a la comunicación entre memorias.

También existen trabajos centrados en reducir la memoria de activaciones mediante compresión. ActNN, propuesto por Chen et al. [2], reduce la huella de memoria del entrenamiento mediante activaciones comprimidas a 2 bits. En una línea similar, DIVISION propone entrenamiento eficiente mediante precisión dual en activaciones [41].

3.5. Herramientas y guías prácticas de rendimiento

Además de los trabajos académicos, existen herramientas y guías orientadas al análisis práctico del rendimiento en PyTorch. TorchBench, presentado por Hao et al. [12], propone un conjunto amplio de modelos para evaluar el rendimiento de PyTorch y cubrir una gran superficie de sus APIs.

La guía de rendimiento de PyTorch elaborada por Migacz [23] recoge recomendaciones prácticas para mejorar el rendimiento de entrenamiento, como el uso de `pin_memory`, `non_blocking`, AMP o técnicas para evitar operaciones innecesarias. Aunque no todas estas recomendaciones están directamente centradas en memoria GPU, forman parte del conjunto de buenas prácticas necesarias para obtener mediciones fiables.

También existen publicaciones orientadas a la reducción de memoria pico. Lightning AI [19] presenta técnicas para reducir el pico de memoria durante entrenamiento en PyTorch, mientras que Gallouédec [10] se centra en visualizar y comprender el uso de memoria GPU en PyTorch.

4. Objetivos y Metodología

4.1. Objetivos

En los últimos años, el crecimiento en tamaño y complejidad de los modelos ha provocado que la memoria disponible en GPU sea uno de los principales factores limitantes durante el entrenamiento. Por ello, es importante comprender cómo se utiliza la memoria y qué técnicas permiten optimizar su consumo.

Este trabajo busca analizar el comportamiento de la memoria GPU durante el entrenamiento de diferentes modelos, estudiando tanto el uso real de memoria por parte de los tensores como la memoria reservada por el *CUDA caching allocator* de PyTorch. Para ello, se implementan y comparan distintas configuraciones de entrenamiento sobre modelos de lenguaje y visión por computador, evaluando el impacto de técnicas como AMP, *gradient checkpointing*, optimizadores de 8 bits, configuraciones del allocator y mecanismos de atención optimizados.

El objetivo principal se divide en los siguientes subobjetivos:

- Revisar los conceptos fundamentales relacionados con la gestión de memoria GPU en PyTorch y CUDA, incluyendo la diferencia entre memoria asignada, memoria reservada, memoria pico y el funcionamiento general del *CUDA caching allocator*.
- Estudiar el funcionamiento de distintas técnicas de optimización de memoria utilizadas durante el entrenamiento de modelos de aprendizaje profundo, como AMP, *gradient checkpointing*, optimizadores de 8 bits y mecanismos de atención eficientes.
- Implementar un entorno experimental aplicado a tres modelos representativos: BERT, GPT-2 y ResNet. Estos modelos permiten analizar distintos patrones de consumo de memoria, al cubrir arquitecturas Transformer de tipo encoder, Transformer autoregresivo y redes convolucionales profundas.
- Comparar distintas configuraciones de entrenamiento para cada modelo, incluyendo una configuración base y varias configuraciones optimizadas.
- Analizar el impacto de las variables de entorno relacionadas con el allocator de PyTorch, como `max_split_size_mb` y `backend:cudaMallocAsync`, evaluando si estas configuraciones reducen la memoria reservada o evitan errores de falta de memoria.
- Generar y analizar *memory snapshots* de PyTorch para estudiar de forma visual el comportamiento del allocator durante el entrenamiento.
- Analizar los resultados obtenidos y extraer conclusiones sobre qué técnicas resultan más efectivas según el tipo de modelo, la configuración de entrenamiento y el patrón de uso de memoria observado.

4.2. Metodología

En esta sección se detalla la metodología seguida, de forma cronológica, para la cumplimentación de los objetivos antes presentados.

Lo primero que se realizó fue la revisión de documentación, artículos y publicaciones relacionadas con la gestión de memoria en PyTorch y CUDA. Se estudiaron conceptos clave como la memoria asignada por tensores activos, la memoria reservada por el *CUDA caching allocator*, los errores de tipo OOM y el uso de herramientas de diagnóstico como `torch.cuda.memory_summary()` y los *memory snapshots*.

Una vez revisados los conceptos básicos, el siguiente paso consistió en estudiar las principales técnicas de optimización de memoria utilizadas durante el entrenamiento de modelos de aprendizaje profundo. En primer lugar, se analizó AMP, una técnica que permite ejecutar determinadas operaciones en precisión reducida para disminuir el consumo de memoria y acelerar el entrenamiento en GPU. También se estudió *gradient checkpointing*, que reduce la memoria necesaria para almacenar activaciones intermedias a cambio de recomputar parte del forward durante el backward. Además, se revisó el uso de optimizadores de 8 bits, especialmente en modelos Transformer, donde los estados internos del optimizador pueden representar una parte importante del consumo total de memoria.

Tras esta revisión inicial, se seleccionaron los modelos que formarían parte del entorno experimental. Se decidió trabajar con tres arquitecturas diferentes: BERT, GPT-2 y ResNet. BERT se eligió como modelo Transformer de tipo encoder aplicado a clasificación de texto; GPT-2 como modelo Transformer autoregresivo para modelado de lenguaje; y ResNet como una red convolucional profunda aplicada a clasificación de imágenes.

El siguiente paso fue implementar los cargadores de datos para cada modelo. Para BERT se utilizó el dataset GLUE/SST-2, orientado a clasificación binaria de sentimiento. Para GPT-2 se utilizó WikiText-2, un dataset habitual en tareas de modelado de lenguaje. Para ResNet se utilizó Tiny ImageNet, adaptando las transformaciones de entrada para probar diferentes resoluciones de imagen. Cada cargador se diseñó de forma independiente para ajustarse a las necesidades de cada arquitectura, pero manteniendo una estructura común en cuanto a batch size, preprocesado y preparación de los tensores de entrada.

Una vez preparados los datasets, se implementaron los modelos y sus correspondientes scripts de entrenamiento. Para cada uno se creó una configuración base y una o varias configuraciones optimizadas. En BERT se compararon configuraciones con y sin AMP, *gradient checkpointing* y optimizador AdamW de 8 bits. En GPT-2 se comparó la atención manual con la función `scaled_dot_product_attention` de PyTorch, además de probar AMP y *gradient checkpointing*. En ResNet se analizaron configuraciones base y optimizadas, así como el efecto de aumentar la resolución de entrada de las imágenes.

Después de implementar los entrenamientos, se desarrolló un sistema de registro de

métricas para obtener información detallada durante cada batch. En cada ejecución se almacenan eventos como el inicio del batch, el final del forward, el final del backward y el final del paso del optimizador. Para cada evento se registra la memoria asignada, la memoria reservada, el pico máximo de memoria y el tiempo de ejecución del batch. Esta información se guarda en ficheros CSV para poder generar posteriormente gráficas comparativas.

A continuación, se implementaron scripts de visualización para representar la evolución de la memoria durante el entrenamiento. Estas gráficas permiten comparar la memoria asignada, la memoria reservada por el allocator y el pico máximo alcanzado. También se generaron versiones ampliadas de los primeros eventos de entrenamiento para observar con mayor detalle cómo varía la memoria durante las fases de forward, backward y actualización del optimizador.

Tras el sistema de medición se diseñaron los experimentos definitivos. En BERT se evaluó el escalado entre BERT-Base y BERT-Large, el efecto de aumentar la longitud de secuencia y el impacto del optimizador AdamW de 8 bits. En GPT-2 se analizó el escalado entre GPT-2 Base, Medium y Large, así como la diferencia entre atención manual y atención optimizada mediante SDPA. En ResNet se estudió el escalado entre ResNet-50 y ResNet-101, el efecto de aumentar la resolución de entrada y el comportamiento de distintas configuraciones del allocator de PyTorch.

Además de los experimentos cuantitativos, se implementó soporte para generar *memory snapshots* de PyTorch. Estos snapshots permiten visualizar el estado interno del allocator y analizar de forma gráfica la evolución temporal de las asignaciones de memoria. Su objetivo dentro del trabajo es complementar el análisis basado en CSV y `memory_summary`, proporcionando una herramienta adicional para depurar ejecuciones concretas y comprender mejor el comportamiento interno del *CUDA caching allocator*. Debido a que el registro del historial de memoria introduce cierta sobrecarga, esta funcionalidad se utiliza como un mecanismo de análisis adicional, no como la fuente principal de comparación entre modelos y configuraciones.

Por último, se analizaron los resultados obtenidos comparando las distintas configuraciones por modelo y por técnica de optimización. El análisis se centra en identificar qué técnicas reducen realmente la memoria asignada, cuáles afectan principalmente a la memoria reservada, qué configuraciones permiten evitar errores de falta de memoria y qué coste temporal introduce cada optimización.

5. Diseño de la Solución

Para este trabajo, se ha desarrollado un entorno experimental con tres modelos, scripts de entrenamiento, cargadores de datos, registro de métricas, generación de gráficas y ejecución en un clúster de cómputo mediante Simple Linux Utility for Resource Management (SLURM). En esta sección se describe la estructura general de la solución, las decisiones de diseño tomadas y la forma en que se han definido los experimentos.

5.1. Visión general del sistema

La solución se basa en una arquitectura modular. Cada modelo dispone de su propia carpeta, que contiene la definición de la arquitectura, el cargador de datos, el script de entrenamiento y el script de generación de gráficas. Además, se ha creado una carpeta común con funcionalidades compartidas, evitando duplicar código y garantizando que las métricas se registren de forma homogénea entre experimentos.

En la Figura 4 se muestra la arquitectura general del entorno experimental desarrollado, incluyendo los modelos implementados, los módulos comunes y las salidas generadas durante las ejecuciones.

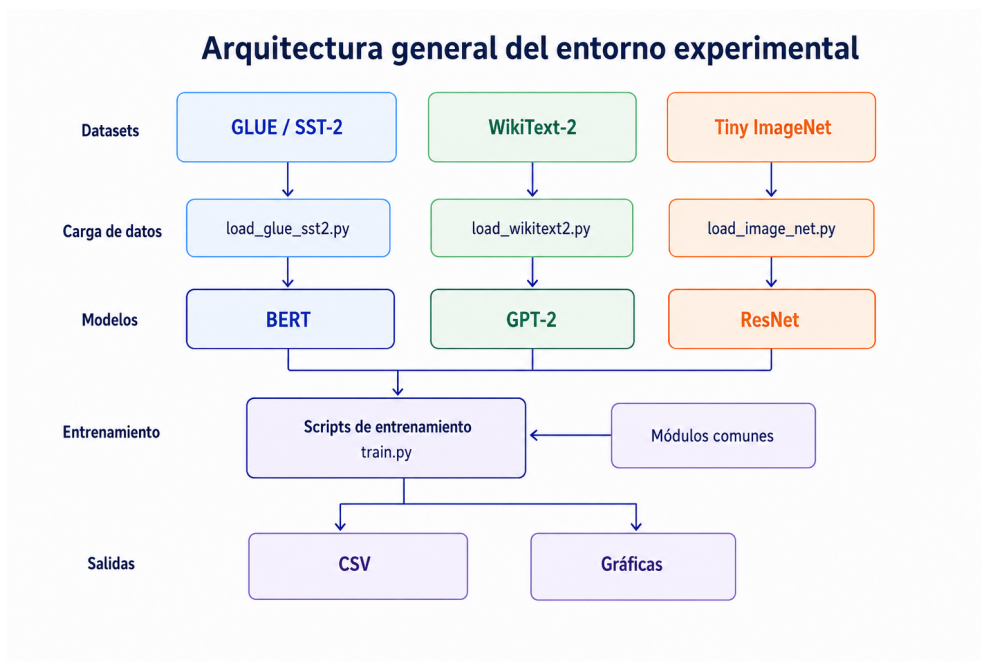


Figura 4: Arquitectura general del entorno desarrollado.

Los modelos seleccionados son BERT, GPT-2 y ResNet. BERT se trata de un modelo

Transformer de tipo encoder utilizado para clasificación de texto [8]; GPT-2 es un modelo Transformer autoregresivo basado en bloques decoder [35]; y ResNet representa una arquitectura convolucional profunda para clasificación de imágenes [13].

El flujo general de ejecución comienza con la carga y preparación del dataset. Después se instancia el modelo, se configura el entrenamiento y se registran métricas de memoria GPU en distintos puntos del batch: antes del *forward*, después del *forward*, después del *backward* y después del paso del optimizador. Finalmente, los datos se almacenan en ficheros CSV y se generan gráficas para analizar la evolución de la memoria asignada, reservada y pico máximo.

5.2. Estructura del proyecto

La estructura del proyecto separa el código específico de cada modelo y el código común utilizado por todos ellos. La organización principal se muestra en la Figura 5.

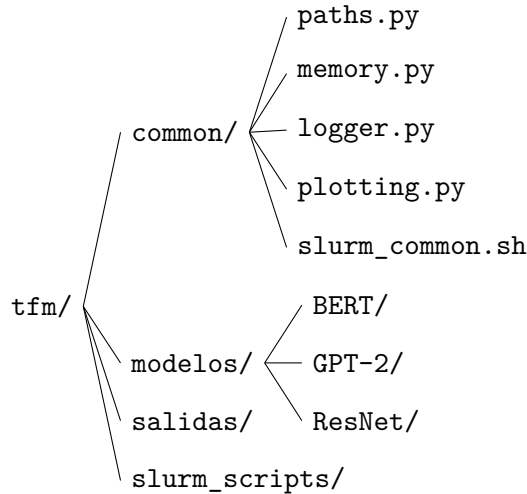


Figura 5: Estructura general del proyecto.

La carpeta **common** contiene los módulos compartidos. **paths.py** centraliza la construcción de rutas de salida, **memory.py** agrupa las funciones relacionadas con memoria GPU, **logger.py** registra las métricas en CSV y **plotting.py** contiene la lógica común para generar gráficas. Cada ejecución almacena sus resultados en una carpeta asociada al nombre del trabajo y a su identificador de SLURM.

5.3. Modelos implementados

5.3.1. BERT

El primer modelo implementado es una versión simplificada de BERT, una arquitectura Transformer encoder. En este trabajo se utiliza para clasificación binaria de texto sobre GLUE/SST-2 [40]. La arquitectura está formada por una capa de embeddings, una pila de capas encoder y un cabezal de clasificación. Cada bloque encoder incluye atención multi-cabeza, una red feed-forward, conexiones residuales y normalización.

Se implementan dos variantes: BERT-Base y BERT-Large. La primera utiliza una dimensión oculta de 768, 12 capas y 12 cabezas de atención. La segunda aumenta la dimensión oculta a 1024, utiliza 24 capas y 16 cabezas. El modelo permite activar *gradient checkpointing*, reduciendo activaciones almacenadas a cambio de recomputar parte del *forward* durante el *backward* [3, 33].

5.3.2. GPT-2

El segundo modelo implementado es una versión simplificada de GPT-2, una arquitectura Transformer autoregresiva orientada a modelado de lenguaje [35]. A diferencia de BERT, GPT-2 utiliza bloques decoder con atención causal, de forma que cada token solo puede atender a posiciones anteriores de la secuencia.

La arquitectura incluye embeddings de tokens y posición, una pila de bloques GPT-2, una normalización final y un cabezal de lenguaje. En la implementación se hay dos variantes de atención: una implementación manual, donde se calcula de forma explícita la matriz de atención, y una versión con `scaled_dot_product_attention`, que permite utilizar kernels optimizados cuando están disponibles [32, 4]. También se permite activar *gradient checkpointing* sobre los bloques del modelo. Se implementan tres tamaños: Base, Medium y Large.

5.3.3. ResNet

El tercer modelo implementado es ResNet, una arquitectura convolucional profunda basada en conexiones residuales [13]. Se utilizan ResNet-50 y ResNet-101, ambas construidas mediante bloques *bottleneck*. A diferencia de los modelos Transformer, su consumo de memoria está más relacionado con la resolución de entrada, la profundidad de la red y el almacenamiento de mapas de activación intermedios.

Cada bloque *bottleneck* está formado por convoluciones 1×1 , 3×3 y 1×1 , junto con una conexión residual. La implementación permite activar *gradient checkpointing* sobre las etapas principales de la red.

5.4. Preparación de datos, entrenamiento y registro de métricas

Cada modelo utiliza un dataset distinto según la tarea que realiza. Para BERT se emplea GLUE/SST-2, un conjunto de datos de clasificación binaria de sentimiento [40]. Las frases se tokenizan con el tokenizador de BERT, aplicando truncamiento y padding hasta una longitud máxima configurable. En los experimentos se utilizan longitudes de 128 y 512 tokens.

Para GPT-2 se utiliza WikiText-2, un dataset habitual en modelado de lenguaje [21]. El texto se tokeniza con el tokenizador de GPT-2 y la secuencia de entrada se desplaza una posición para construir los objetivos, de forma que el modelo aprende a predecir el siguiente token.

Para ResNet se utiliza Tiny ImageNet, un subconjunto reducido de ImageNet compuesto por 200 clases [18]. Las imágenes se cargan mediante `ImageFolder` y se aplican transformaciones de redimensionado, recorte centrado, conversión a tensor y normalización. Además de la resolución estándar, se realizan experimentos con imágenes de 512×512 píxeles para aumentar la presión de memoria.

Cada modelo cuenta con su propio script de entrenamiento, aunque todos siguen una estructura común. Primero se procesan los argumentos de entrada, después se carga el dataset, se instancia el modelo y se configura el optimizador. Durante el entrenamiento se registran métricas en distintos puntos del batch, permitiendo analizar cómo evoluciona la memoria durante el *forward*, el *backward* y la actualización de parámetros.

En BERT y GPT-2 se utiliza AdamW como optimizador principal, una opción habitual en modelos Transformer [16, 20]. En BERT se incluye además AdamW de 8 bits mediante `bitsandbytes`, con el objetivo de reducir la memoria asociada a los estados internos del optimizador [5, 1]. En ResNet se utiliza SGD con momentum y weight decay.

Los scripts permiten activar AMP mediante `autocast` y `GradScaler`. Esta técnica ejecuta determinadas operaciones en precisión reducida y ayuda a evitar problemas de subdesbordamiento en los gradientes durante el entrenamiento en FP16 [22, 27]. También se permite activar *gradient checkpointing*, reduciendo memoria de activaciones a cambio de recomputación.

Una parte importante del diseño es el sistema de registro de métricas. Para cada batch se almacenan varios eventos: `batch_start`, `after_forward`, `after_backward`, `after_step` y, en caso de fallo por falta de memoria, `OOM_CRASH`. Para cada evento se registran la época, el batch, el tipo de evento, la memoria asignada, la memoria reservada, el pico máximo de memoria y el tiempo del batch.

La memoria asignada representa la memoria ocupada por tensores activos, mientras que la memoria reservada incluye bloques mantenidos por el *CUDA caching allocator*

para reutilización posterior [28, 7]. Por ello, una ejecución puede mostrar una memoria reservada elevada aunque la memoria realmente ocupada por tensores sea menor.

El tiempo de entrenamiento se mide mediante eventos CUDA, evitando errores derivados de la ejecución asíncrona de operaciones en GPU. Para calcular el tiempo medio por batch se utilizan únicamente los eventos `after_step`, ya que representan el final completo de una iteración.

A partir de los CSV se generan gráficas de rendimiento para cada ejecución. Estas muestran la evolución de la memoria asignada, la memoria reservada y el pico máximo de memoria. También se genera una versión ampliada de los primeros eventos para observar el comportamiento inicial del allocator y detectar patrones como crecimiento progresivo de memoria reservada o diferencias entre memoria asignada y reservada.

5.5. Diagnóstico de memoria y configuraciones del allocator

Además del registro cuantitativo mediante CSV, se implementa soporte para generar *memory snapshots* de PyTorch. Estos snapshots permiten visualizar el estado interno del allocator, incluyendo bloques activos, bloques reservados, eventos de asignación y liberación de memoria y trazas asociadas a las operaciones que generaron dichas asignaciones [29, 38, 6].

Para capturarlos se activa el historial de memoria antes de crear el modelo y ejecutar el entrenamiento. Tras esto se guarda un fichero `pickle`, que puede cargarse en el visualizador de memoria de PyTorch [31]. Esta funcionalidad se plantea como complemento visual al análisis principal. Los CSV permiten comparar métricas de forma homogénea, mientras que los snapshots facilitan la depuración y el análisis detallado de ejecuciones concretas.

Uno de los objetivos del trabajo es estudiar el comportamiento del *CUDA caching allocator*. Para ello, se prueban configuraciones mediante la variable de entorno `PYTORCH\_CUDA\_\_ALLOC\_CONF`. En particular, se evalúan `max_split_size_mb:128`, que limita el tamaño máximo de los bloques que el allocator puede dividir, y `backend:cudaMallocAsync`, que permite utilizar el allocator asíncrono de CUDA basado en memoria ordenada por stream [24, 17].

Estas configuraciones se prueban en ResNet-101 con imágenes de 512×512 , ya que esta configuración genera una presión de memoria elevada y permite observar con mayor claridad el comportamiento del allocator.

5.6. Ejecución con SLURM y experimentos definidos

Los experimentos se ejecutan en clúster de GACOP de la Universidad de Murcia mediante scripts SLURM. Cada script define el nombre del job, la partición de ejecución,

los ficheros de salida y error, y el entorno de ejecución dentro de un contenedor Singularity. Para mantener una organización clara, cada ejecución genera una carpeta de salida con el nombre del trabajo seguido del ID del mismo. Dentro de esta carpeta se almacenan logs, CSV, gráficas y diagnósticos de memoria. Además, se crea un script común, **slurm_common.sh**, que centraliza la carga de módulos, la definición de rutas, la selección del contenedor, la activación del entorno virtual y la redirección de logs.

A partir de la infraestructura anterior se definen experimentos para cada modelo. En BERT se compara el escalado entre BERT-Base y BERT-Large, el efecto de la longitud de secuencia, el uso de AMP y *gradient checkpointing*, y el impacto de AdamW de 8 bits. En GPT-2 se analiza el escalado entre GPT-2 Base, Medium y Large, además de comparar atención manual con SDPA. En ResNet se estudia el escalado entre ResNet-50 y ResNet-101, el impacto de aumentar la resolución de entrada y el comportamiento de las configuraciones del allocator. Con esta estructura, el sistema permite realizar una comparación homogénea entre modelos y optimizaciones.

6. Análisis de Resultados

En esta sección se presentan y analizan los resultados obtenidos durante la ejecución de los experimentos. El objetivo es comprobar qué configuración consume menos memoria, así como interpretar cómo se comporta cada técnica de optimización según el tipo de modelo utilizado. Para ello se comparan los modelos BERT, GPT-2 y ResNet.

Los experimentos estudian el escalado de memoria al aumentar el tamaño del modelo o la entrada, el impacto de técnicas como AMP, *gradient checkpointing*, AdamW de 8 bits y SDPA, y el comportamiento del *CUDA caching allocator* mediante configuraciones como `max_split_size_mb` y `cudaMallocAsync`.

6.1. Entorno experimental y métricas

Las ejecuciones se realizaron en el clúster de GACOP de la Universidad de Murcia mediante scripts SLURM, utilizando contenedores Singularity y entornos virtuales de Python preparados para los experimentos. Cada ejecución genera sus propios ficheros de salida, errores, CSV de métricas, gráficas y diagnósticos de memoria. La GPU utilizada fue una NVIDIA GeForce RTX 4090 con soporte CUDA y 24 GB de memoria.

Las métricas principales son memoria asignada, memoria reservada, memoria pico y tiempo medio por batch. La memoria asignada representa la memoria ocupada por tensores activos, mientras que la memoria reservada corresponde a la memoria gestionada por el *CUDA caching allocator*. Hay que tener en cuenta esta diferencia, ya que una optimización puede reducir la memoria realmente utilizada sin modificar en la misma proporción

la memoria reservada [28, 7].

El tiempo medio por batch se calcula únicamente a partir de los eventos `after_step`, que representan el final completo de una iteración. Las gráficas generadas muestran la evolución de la memoria asignada, reservada y pico máximo, permitiendo observar el comportamiento del entrenamiento durante los primeros batches y detectar diferencias entre memoria usada y memoria mantenida por el allocator.

6.2. Resultados en BERT

Los experimentos con BERT analizan el escalado entre BERT-Base y BERT-Large, el impacto de aumentar la longitud de secuencia de 128 a 512 tokens y el efecto de AMP, *gradient checkpointing* y AdamW de 8 bits. Todos los experimentos se ejecutan con batch size 32.

Tabla 1: Resultados obtenidos en los experimentos con BERT.

Modelo	Configuración	Long.	Mem. pico (MB)	Mem. reservada (MB)	Tiempo (ms)
BERT-Base	Base	128	4693.90	5038.00	187.12
BERT-Large	Base	128	12794.26	13084.00	349.75
BERT-Large	AMP + checkpointing	128	7619.84	7848.00	259.76
BERT-Large	Base	512	OOM	22852.00	–
BERT-Large	AMP + checkpointing	512	7629.78	9528.00	629.88
BERT-Large	AdamW FP32	128	12794.26	13084.00	355.88
BERT-Large	AdamW 8-bit	128	10906.21	11102.00	355.44
BERT-Large	AdamW 8-bit	512	OOM	22852.00	–

En la Figura 6 se muestra la comparación visual de la memoria pico media obtenida en las configuraciones de BERT ejecutadas correctamente.

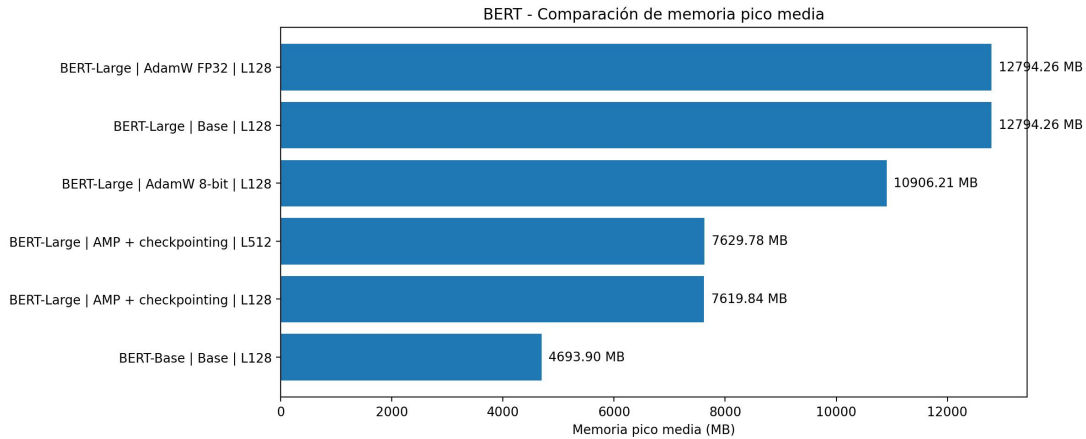


Figura 6: Comparación de memoria pico media en los experimentos con BERT.

La comparación entre BERT-Base y BERT-Large con longitud 128 muestra el efecto directo del tamaño del modelo. BERT-Base alcanza 4693.90 MB de memoria pico, mientras que BERT-Large alcanza 12794.26 MB, aproximadamente 2.7 veces más. El tiempo medio por batch también aumenta de 187.12 ms a 349.75 ms, debido al incremento de capas, dimensión oculta y cabezas de atención.

La combinación de AMP y *gradient checkpointing* reduce la memoria en BERT-Large con longitud 128, pasando de 12794.26 MB a 7619.84 MB, una reducción aproximada del 40.44 %. Además, el tiempo disminuye de 349.75 ms a 259.76 ms. Aunque el checkpointing introduce recomputación, en este caso el uso de AMP compensa ese coste y mejora el rendimiento.

El aumento de longitud de secuencia tiene un impacto muy elevado, debido a la naturaleza de los modelos Transformer, donde la atención escala de forma cuadrática con respecto a la longitud de secuencia. BERT-Large con longitud 512 produce OOM en configuración base, mientras que la configuración optimizada sí se ejecuta con 7629.78 MB de memoria pico. No obstante, el tiempo aumenta hasta 629.88 ms, mostrando que procesar secuencias largas sigue siendo costoso aunque la memoria pico quede controlada.

AdamW de 8 bits reduce la memoria pico de BERT-Large con longitud 128 de 12794.26 MB a 10906.21 MB, una mejora aproximada del 14.76 %, sin modificar apenas el tiempo. Esta optimización reduce la memoria asociada a los estados internos del optimizador [5, 1], pero no actúa sobre las activaciones. Por ello, no evita el OOM en BERT-Large con longitud 512.

En resumen, para BERT el tamaño del modelo y la longitud de secuencia condicionan fuertemente la memoria. AMP y *gradient checkpointing* son las optimizaciones más efectivas, mientras que AdamW de 8 bits ofrece un ahorro adicional más limitado.

6.3. Resultados en GPT-2

Los experimentos con GPT-2 estudian el escalado entre configuraciones Base, Medium y Large, el impacto de AMP y *gradient checkpointing*, y el uso de SDPA frente a una atención manual. Todos los experimentos utilizan longitud de secuencia 512 y batch size 8.

Tabla 2: Resultados obtenidos en los experimentos con GPT-2.

Modelo	Configuración	Long.	Mem. pico (MB)	Mem. reservada (MB)	Tiempo (ms)
GPT-2 Base	Base	512	9507.14	10160.00	226.05
GPT-2 Medium	Base	512	20275.98	21178.00	457.74
GPT-2 Medium	AMP + checkpointing	512	8514.44	10628.00	378.38
GPT-2 Large	Base	512	OOM	22972.00	–
GPT-2 Large	AMP + checkpointing	512	18349.68	20872.00	639.57
GPT-2 Large	SDPA	512	OOM	23004.00	–
GPT-2 Large	SDPA + AMP + checkpointing	512	18342.90	20518.00	438.43

En la Figura 7 se muestra la comparación visual de la memoria pico media obtenida en las configuraciones de GPT-2 ejecutadas correctamente.

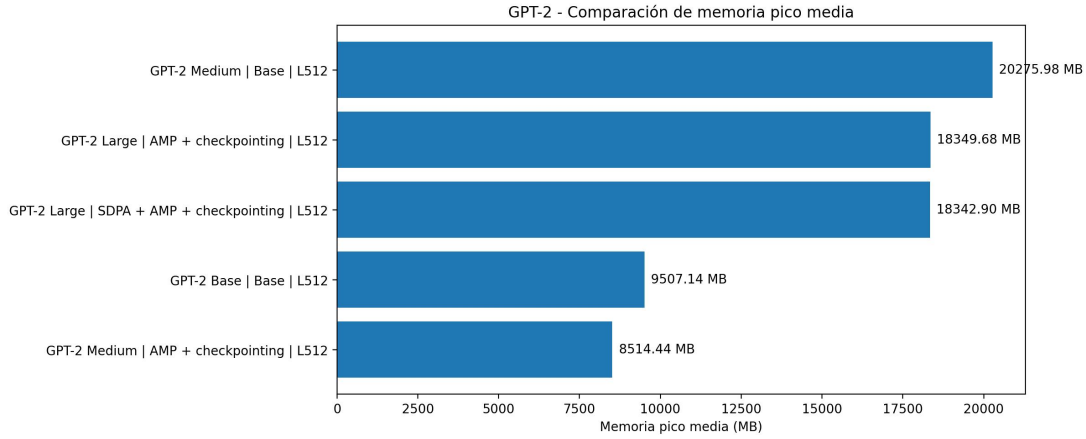


Figura 7: Comparación de memoria pico media en los experimentos con GPT-2.

El escalado del modelo tiene un impacto elevado, GPT-2 Base alcanza 9507.14 MB de memoria pico, mientras que GPT-2 Medium alcanza 20275.98 MB, un incremento aproximado del 113.27 %. El tiempo también aumenta de 226.05 ms a 457.74 ms. GPT-2 Large no completa el primer batch en configuración base, mostrando que no es viable con longitud 512 y batch size 8 sin optimizaciones.

AMP y *gradient checkpointing* reducen de forma notable la memoria en GPT-2 Medium, pasando de 20275.98 MB a 8514.44 MB. Esto supone una reducción aproximada del 58.01 %. Además, el tiempo baja de 457.74 ms a 378.38 ms. En GPT-2 Large, esta combinación permite ejecutar una configuración que en modo base terminaba en OOM, con una memoria pico de 18349.68 MB.

SDPA por sí solo no evita el OOM en GPT-2 Large, lo que indica que el problema de memoria no está únicamente en la implementación de la atención. Sin embargo, al combinar SDPA con AMP y *gradient checkpointing*, el tiempo medio baja de 639.57 ms a

438.43 ms, una mejora aproximada del 31.45 %. La memoria pico apenas cambia: 18349.68 MB sin SDPA frente a 18342.90 MB con SDPA.

Este resultado indica que SDPA sí mejora la eficiencia computacional, aunque el pico de memoria global parece estar determinado por otros componentes del entrenamiento, como parámetros, gradientes, estados del optimizador o activaciones de capas feed-forward. Además, al utilizar AMP y *gradient checkpointing*, parte del ahorro potencial sobre activaciones queda solapado con estas técnicas [32, 30, 15].

En la Figura 8 se muestra la evolución completa de la memoria durante la ejecución de GPT-2 Large con SDPA, AMP y *gradient checkpointing*. La Figura 9 muestra una ampliación de los primeros eventos para observar con mayor detalle el comportamiento inicial.

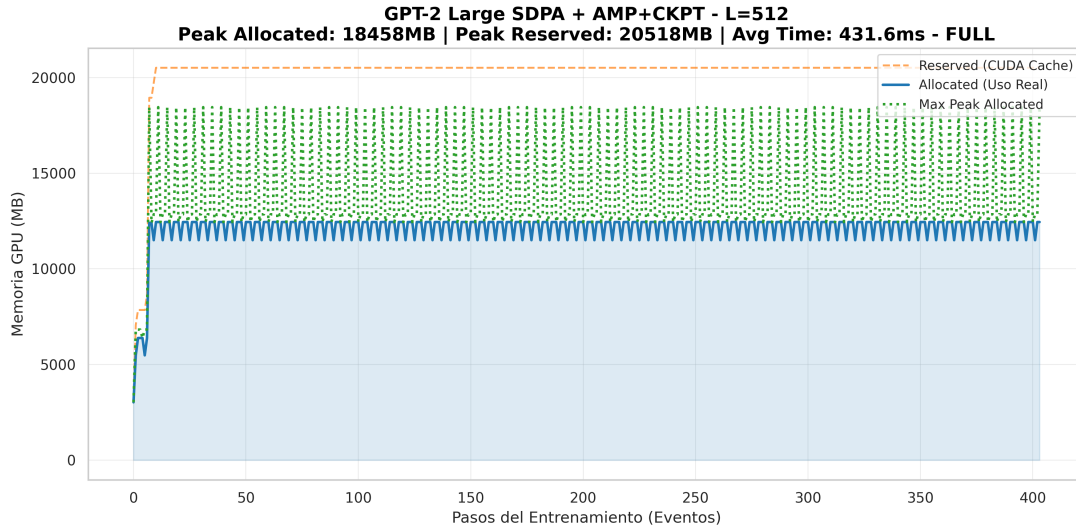


Figura 8: Evolución completa de la memoria en GPT-2 Large con SDPA, AMP y *gradient checkpointing*.

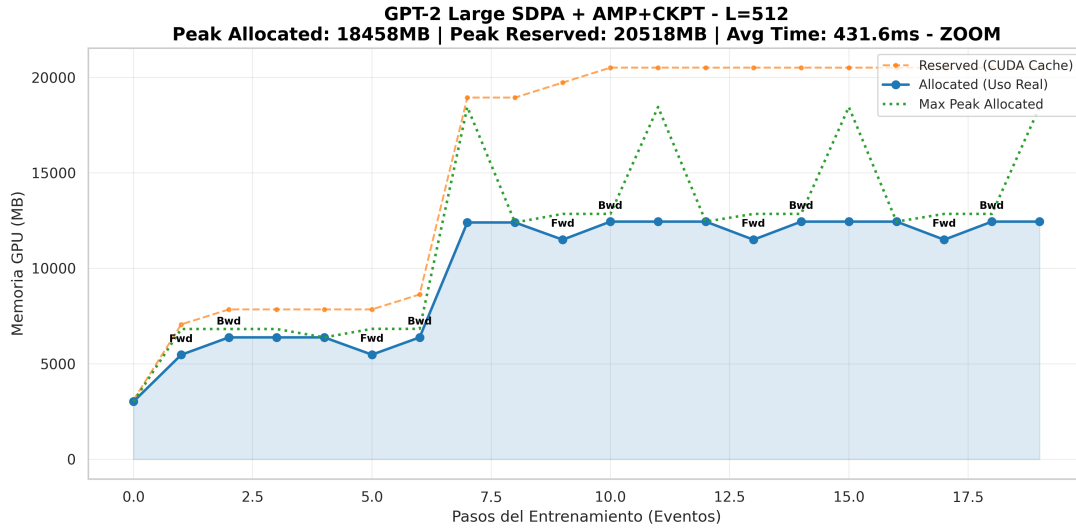


Figura 9: Detalle de los primeros eventos de memoria en GPT-2 Large con SDPA, AMP y *gradient checkpointing*.

En el caso de GPT-2 el aumento de tamaño del modelo eleva rápidamente el consumo de memoria. AMP y *gradient checkpointing* son esenciales para evitar OOM en GPT-2 Large, mientras que SDPA aporta principalmente una mejora temporal.

6.4. Resultados en ResNet

Los experimentos con ResNet estudian el escalado entre ResNet-50 y ResNet-101, el impacto de aumentar la resolución de entrada de 224×224 a 512×512 píxeles y el efecto de distintas configuraciones del allocator. Todos los experimentos se ejecutan con batch size 64.

Tabla 3: Resultados obtenidos en los experimentos con ResNet.

Modelo	Configuración	Res.	Mem. pico (MB)	Mem. reservada (MB)	Tiempo (ms)
ResNet-50	Base	224	5584.04	6144.00	169.82
ResNet-50	AMP + checkpointing	224	1844.88	2054.00	177.71
ResNet-101	Base	224	8247.79	9062.00	202.07
ResNet-101	AMP + checkpointing	224	2856.76	3100.00	224.01
ResNet-101	Base	512	OOM	22760.00	–
ResNet-101	AMP + checkpointing	512	12580.04	15734.00	689.88
ResNet-101	max_split_size_mb=128	512	OOM	22764.00	–
ResNet-101	max_split_size_mb=128 + AMP + checkpointing	512	12583.46	21278.00	685.42
ResNet-101	cudaMallocAsync	512	OOM	22752.00	–
ResNet-101	cudaMallocAsync + AMP + checkpointing	512	12574.14	14208.00	687.89

En la Figura 10 se muestra la comparación visual de la memoria pico media obtenida en las configuraciones de ResNet ejecutadas correctamente.

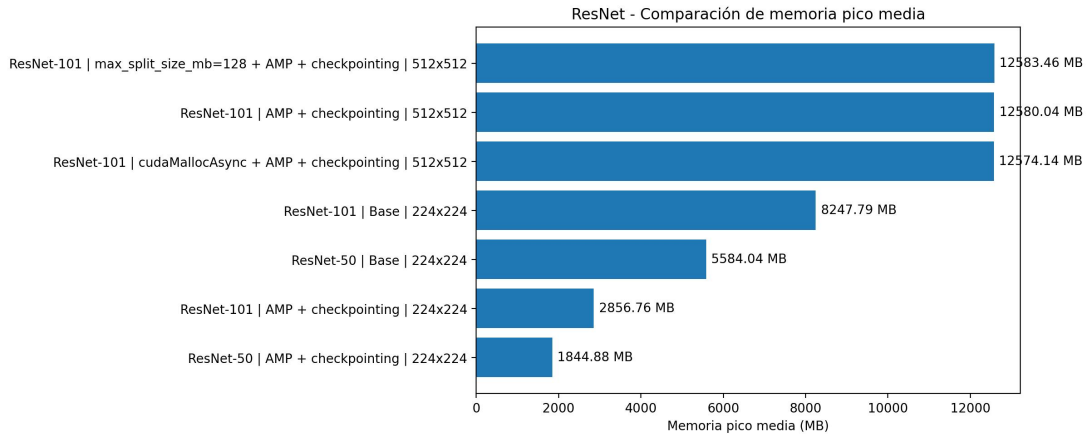


Figura 10: Comparación de memoria pico media en los experimentos con ResNet.

Al comparar ResNet-50 y ResNet-101 con resolución 224×224 , la memoria pico aumenta de 5584.04 MB a 8247.79 MB, un incremento aproximado del 47.70 %. El tiempo también sube de 169.82 ms a 202.07 ms. Este comportamiento se debe a la mayor profundidad de ResNet-101 y al incremento de activaciones intermedias.

AMP y *gradient checkpointing* reducen de forma muy significativa la memoria en ambas arquitecturas. En ResNet-50, la memoria pico baja de 5584.04 MB a 1844.88 MB, y en ResNet-101 pasa de 8247.79 MB a 2856.76 MB. En ambos casos la reducción supera el 65 %. Sin embargo, el tiempo aumenta ligeramente, debido al coste de recomputación que hace el checkpointing.

El aumento de resolución tiene un impacto muy grande, ResNet-101 con imágenes de 512×512 produce OOM en configuración base, mientras que la versión optimizada se ejecuta con 12580.04 MB de memoria pico. En comparación con ResNet-101 optimizada a 224×224 , la memoria pico aumenta más de un 300 % y el tiempo pasa de 224.01 ms a 689.88 ms.

Las configuraciones del allocator no evitan el OOM cuando se aplican sin AMP ni checkpointing. Tanto `max_split_size_mb=128` como `cudaMallocAsync` fallan en ResNet-101 a 512×512 . En configuraciones optimizadas, `max_split_size_mb=128` mantiene una memoria pico similar, pero aumenta la memoria reservada de 15734 MB a 21278 MB. En cambio, `cudaMallocAsync` mantiene la memoria pico prácticamente igual y reduce la memoria reservada a 14208 MB. Estas configuraciones modifican principalmente la gestión interna de bloques reservados, pero no reducen la memoria realmente necesaria

para ejecutar el modelo. `cudaMallocAsync` muestra un efecto moderado sobre la memoria reservada, mientras que `max_split_size_mb=128` no resulta beneficioso en este escenario.

En ResNet la profundidad y la resolución de entrada condicionan fuertemente la memoria. AMP y *gradient checkpointing* permiten ejecutar configuraciones exigentes, aunque con un aumento en el coste temporal moderado. Las configuraciones del allocator pueden afectar a la memoria reservada, pero no solucionan por sí solas un consumo real excesivo.

6.5. Comparación global de optimizaciones

Cada optimización actúa sobre una parte distinta del entrenamiento. AMP modifica el tipo de dato de determinadas operaciones, *gradient checkpointing* reduce activaciones almacenadas, AdamW de 8 bits reduce estados del optimizador, SDPA optimiza la atención y las configuraciones del allocator modifican la reserva y reutilización de bloques.

La técnica con mayor impacto general es la combinación de AMP y *gradient checkpointing*. En BERT-Large reduce la memoria pico de 12794.26 MB a 7619.84 MB; en GPT-2 Medium, de 20275.98 MB a 8514.44 MB; y en ResNet-101, de 8247.79 MB a 2856.76 MB. Además, permite ejecutar configuraciones que en modo base producían OOM, como BERT-Large con longitud 512, GPT-2 Large con longitud 512 y ResNet-101 con imágenes de 512×512 .

El efecto temporal depende del modelo. En BERT y GPT-2, AMP compensa el coste de recomputación del checkpointing y reduce el tiempo por batch. En ResNet, el tiempo aumenta ligeramente, aunque el ahorro de memoria es muy elevado. Esto muestra que el equilibrio entre reducción de memoria y coste computacional depende del tipo de arquitectura.

En las Figuras 11, 12 y 13 se compara el tiempo medio por batch para las configuraciones principales de cada modelo.

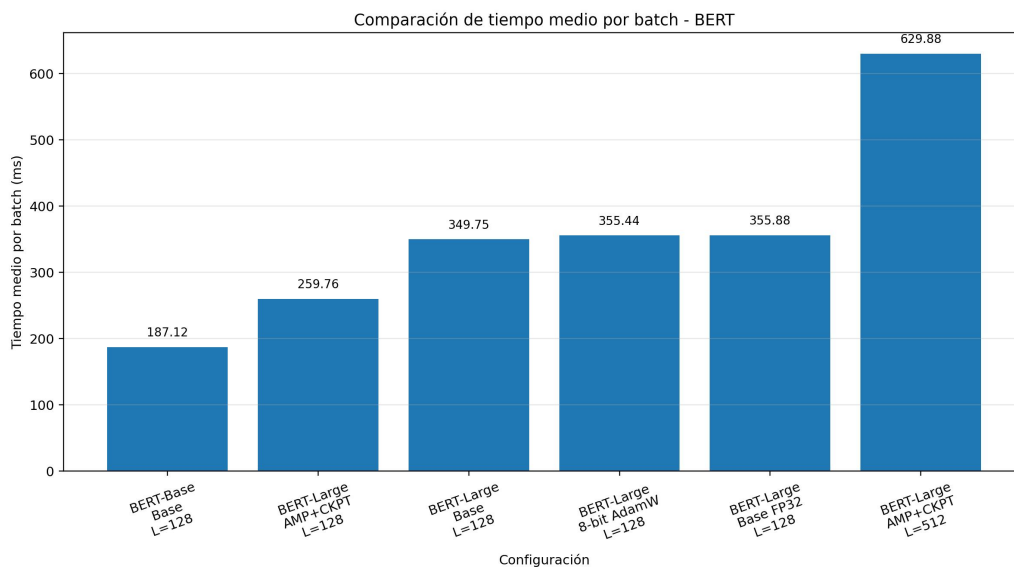


Figura 11: Comparación del tiempo medio por batch en los experimentos con BERT.

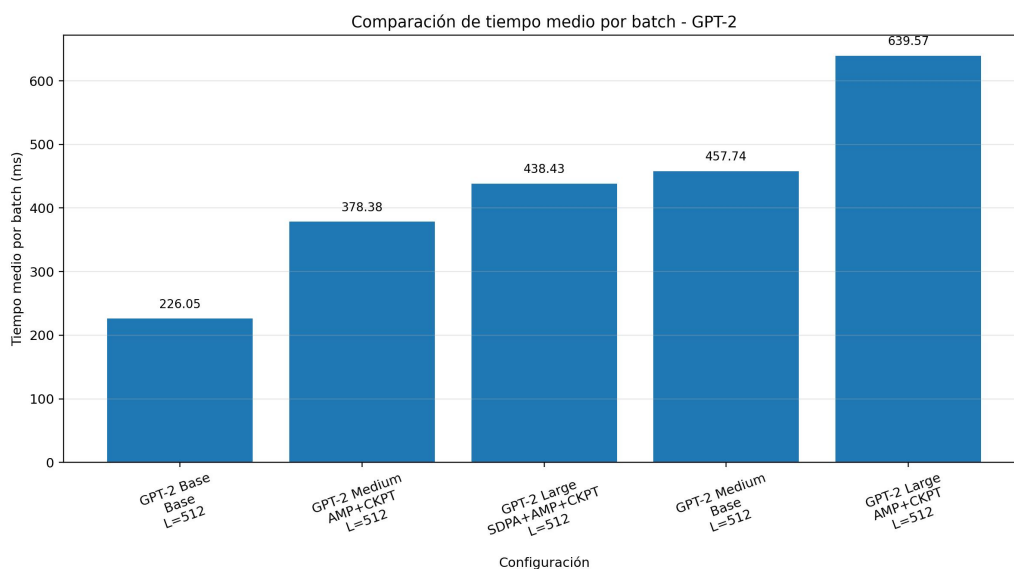


Figura 12: Comparación del tiempo medio por batch en los experimentos con GPT-2.

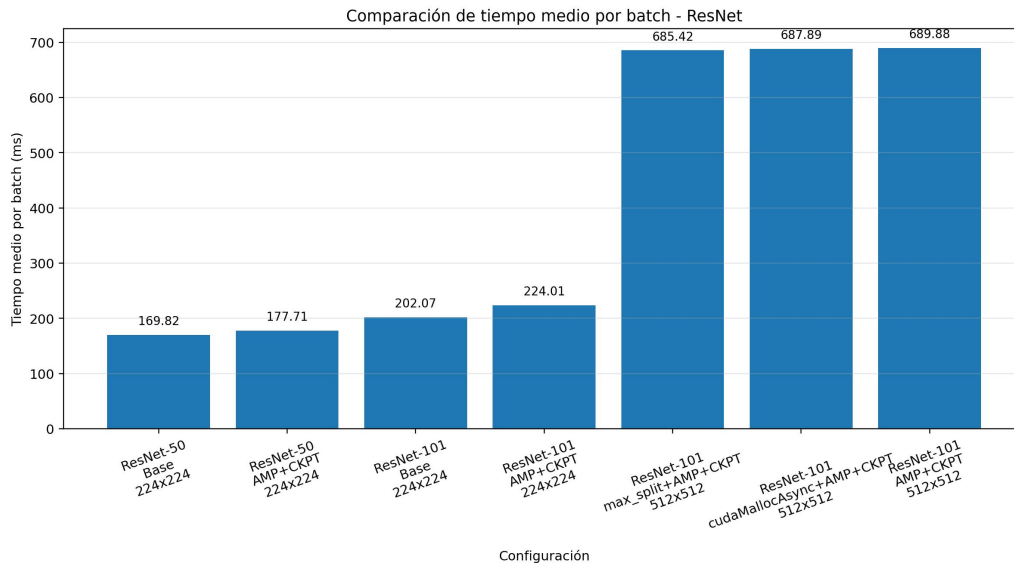


Figura 13: Comparación del tiempo medio por batch en los experimentos con ResNet.

AdamW de 8 bits reduce memoria de forma moderada en BERT, pero no resuelve escenarios donde el cuello de botella principal son las activaciones. SDPA mejora claramente el tiempo en GPT-2 Large, aunque no reduce de forma apreciable la memoria pico al combinarse con AMP y checkpointing, esto muestra que la atención no domina por sí sola el pico global en esa configuración.

Las configuraciones del allocator muestran que modificar la política de reserva de memoria no garantiza una reducción del consumo real. En ResNet-101 a 512×512 , tanto `max_split_size_mb=128` como `cudaMallocAsync` producen OOM sin optimizaciones. En las versiones optimizadas, `cudaMallocAsync` reduce parcialmente la memoria reservada, mientras que `max_split_size_mb=128` la aumenta[28, 7, 24].

6.6. Discusión general

Los resultados confirman que no existe una única técnica que mejore todos los aspectos del entrenamiento en GPU. En modelos Transformer, la memoria se ve muy afectada por la longitud de secuencia, el número de capas y la dimensión interna. En ResNet, el factor más determinante es la resolución de entrada, debido al tamaño de los mapas de activación. También se observa que algunas optimizaciones tienen más impacto en memoria y otras en tiempo. AMP y *gradient checkpointing* son las técnicas más efectivas para reducir memoria, especialmente cuando el problema principal son las activaciones. AdamW de 8 bits reduce los estados del optimizador, pero su efecto es menor. SDPA mejora el rendimiento temporal en GPT-2, aunque el pico de memoria depende del conjunto completo del

entrenamiento. Las configuraciones del allocator pueden modificar la memoria reservada, pero no sustituyen a técnicas que reducen el consumo real del modelo.

7. Conclusiones y Vías Futuras

La memoria GPU es uno de los recursos más importantes durante el entrenamiento de modelos de aprendizaje profundo. Aunque las GPU actuales ofrecen una gran capacidad de cómputo, la memoria disponible sigue siendo un factor limitante, especialmente al trabajar con modelos grandes, secuencias largas, imágenes de alta resolución o batches elevados. Es por esto que comprender cómo PyTorch gestiona la memoria mediante CUDA y su *CUDA caching allocator* es fundamental para interpretar correctamente el comportamiento de los entrenamientos.

Este trabajo ha estudiado la gestión de memoria GPU durante el entrenamiento de modelos en PyTorch y CUDA. Para ello, se ha desarrollado un entorno experimental compuesto por tres modelos representativos: BERT, GPT-2 y ResNet. Cada uno de ellos permite analizar un patrón de memoria distinto. BERT es un modelo Transformer de tipo encoder aplicado a clasificación de texto; GPT-2 se trata un modelo Transformer autoregresivo con atención causal; y ResNet consiste en una arquitectura convolucional profunda aplicada a clasificación de imágenes. Esta selección permite comparar el comportamiento de distintas técnicas de optimización en modelos de naturaleza diferente.

El sistema implementado incluye cargadores de datos, modelos, scripts de entrenamiento, generación de gráficas, scripts de ejecución en SLURM y una capa común para registrar métricas de memoria y rendimiento. En cada ejecución se han medido la memoria asignada, la memoria reservada, el pico máximo de memoria y el tiempo medio por batch. Es importante distinguir entre memoria reservada y memoria asignada, ya que la memoria asignada representa la memoria ocupada por tensores activos, mientras que la memoria reservada corresponde a la memoria gestionada por el allocator de PyTorch. Una reducción en la memoria realmente utilizada por el modelo no siempre implica una reducción proporcional de la memoria reservada.

Los resultados muestran que la combinación de AMP y *gradient checkpointing* ha sido la optimización más efectiva de forma general. En BERT-Large con longitud de secuencia 128, esta configuración reduce la memoria pico de 12794.26 MB a 7619.84 MB. En GPT-2 Medium, la memoria pico pasa de 20275.98 MB a 8514.44 MB. En ResNet-101 con resolución 224×224 , la memoria pico se reduce de 8247.79 MB a 2856.76 MB. Además, esta combinación permite ejecutar configuraciones que en modo base terminaban en OOM, como BERT-Large con longitud 512, GPT-2 Large con longitud 512 y ResNet-101 con imágenes de 512×512 .

Sin embargo, el efecto temporal de esta optimización no ha sido igual en todos los

modelos. En BERT y GPT-2, AMP y *gradient checkpointing* reducen tanto la memoria como el tiempo medio por batch. En cambio, en ResNet se observa un ligero aumento del tiempo de entrenamiento. Esto se debe a que el *gradient checkpointing* reduce memoria a costa de recomputar parte del *forward* durante el *backward*. En modelos Transformer, el beneficio de AMP compensa mejor este coste, mientras que en ResNet el ahorro de memoria se obtiene con un pequeño coste temporal adicional.

También se ha evaluado el uso de AdamW de 8 bits en BERT. Esta técnica reduce la memoria pico de BERT-Large con longitud 128 de 12794.26 MB a 10906.21 MB, sin modificar de forma significativa el tiempo de entrenamiento. Esto confirma que los optimizadores de 8 bits son útiles para reducir la memoria asociada a los estados internos del optimizador. No obstante, su efecto es más limitado que el de AMP y *gradient checkpointing*, ya que no actúa directamente sobre las activaciones intermedias.

En GPT-2 se analizó además el uso de `scaled_dot_product_attention`. Los resultados muestran que SDPA apenas modifica la memoria pico cuando se combina con AMP y *gradient checkpointing*, pero sí reduce de forma clara el tiempo de ejecución. En GPT-2 Large, el tiempo medio por batch pasa de 639.57 ms a 438.43 ms al utilizar SDPA junto con el resto de optimizaciones. Esto indica que la implementación optimizada de atención mejora la eficiencia computacional, aunque el pico de memoria global lo definen otros elementos del entrenamiento.

En ResNet se estudió el impacto de las configuraciones del allocator mediante `max_split_size_mb=128` y `cudaMallocAsync`. Los resultados muestran que estas configuraciones no resuelven por sí solas los errores OOM cuando el consumo real del modelo supera la capacidad disponible de la GPU. En ResNet-101 con resolución 512×512 , ambas configuraciones siguen produciendo OOM si no se aplican AMP y *gradient checkpointing*. En las configuraciones optimizadas, `cudaMallocAsync` reduce parcialmente la memoria reservada, mientras que `max_split_size_mb=128` incluso aumenta la memoria reservada. Modificar el allocator puede afectar a la memoria reservada, pero no reduce necesariamente la memoria realmente necesaria para entrenar el modelo.

Durante el desarrollo también se implementó soporte para generar *memory snapshots* de PyTorch. Esta herramienta permite visualizar de forma más detallada el comportamiento interno del allocator, incluyendo bloques activos, memoria reservada y eventos de asignación y liberación. En este trabajo, los snapshots se plantean como una funcionalidad complementaria de diagnóstico y visualización, no como la fuente principal de resultados. El análisis final se basa en los CSV generados durante el entrenamiento y en los resúmenes textuales obtenidos mediante `torch.cuda.memory_summary()`, que permiten realizar una comparación homogénea entre modelos, configuraciones y técnicas de optimización.

En conjunto, los resultados confirman que la optimización de memoria GPU no depende de una única técnica, cada optimización actúa sobre una parte distinta del entrenamiento.

AMP reduce el coste de determinadas operaciones mediante precisión reducida, *gradient checkpointing* disminuye las activaciones almacenadas, AdamW de 8 bits reduce los estados del optimizador, SDPA mejora el cálculo de la atención y las configuraciones del allocator modifican la forma en que PyTorch reserva memoria. Por tanto, para interpretar correctamente los resultados es necesario analizar varias métricas al mismo tiempo: memoria asignada, memoria reservada, memoria pico y tiempo de ejecución.

Como trabajo futuro, se plantean las siguientes líneas de investigación. En primer lugar, sería interesante ampliar los experimentos con nuevas configuraciones de batch size y longitud de secuencia. Esto permitiría analizar con más detalle en qué escenarios las distintas optimizaciones mejoran las métricas de memoria y tiempo de entrenamiento.

Otra línea futura consiste en incorporar nuevas técnicas de optimización no implementadas en este trabajo. Por ejemplo, podrían evaluarse métodos de *offloading* entre CPU y GPU, técnicas de compresión de activaciones como ActNN o DIVISION, o estrategias de entrenamiento distribuido como ZeRO. Estas técnicas permitirían estudiar escenarios más cercanos al entrenamiento de modelos de gran escala, aunque requerirían una infraestructura más compleja. También sería interesante repetir los experimentos en otras GPU y con versiones diferentes de CUDA y PyTorch. Esto permitiría comprobar si los resultados observados se mantienen en otros entornos hardware y software.

Por último, se podría ampliar el entorno experimental incorporando más modelos y tareas. En visión por computador podrían probarse arquitecturas más recientes, como Vision Transformers. En procesamiento de lenguaje natural podrían evaluarse modelos de mayor tamaño o arquitecturas encoder-decoder. Esto permitiría estudiar si las conclusiones obtenidas con BERT, GPT-2 y ResNet se mantienen en modelos más actuales y con patrones de memoria diferentes.

En conclusión, este trabajo muestra que una gestión adecuada de la memoria GPU es esencial para entrenar modelos de aprendizaje profundo de forma eficiente. No basta con observar la memoria total ocupada en la GPU, sino que es necesario entender qué parte corresponde a tensores activos, qué parte está reservada por el allocator y en qué momento se alcanza el pico de memoria. A partir de este análisis, técnicas como AMP, *gradient checkpointing*, optimizadores de 8 bits, SDPA y configuraciones del allocator pueden aplicarse de forma más razonada, seleccionando la más adecuada según el modelo y el problema concreto.

Referencias

- [1] bitsandbytes Contributors. 8-bit optimizers. bitsandbytes Documentation, n.d.
- [2] Jianfei Chen, Lianmin Zheng, Zhewei Yao, Dequan Wang, Ion Stoica, Michael W. Mahoney, and Joseph E. Gonzalez. Actnn: Reducing training memory footprint via 2-bit activation compressed training. *CoRR*, abs/2104.14129, 2021.
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *CoRR*, abs/1604.06174, 2016.
- [4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.
- [5] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization. *CoRR*, abs/2110.02861, 2021.
- [6] Zachary DeVito. Debugging PyTorch memory use with snapshots. Zach DeVito’s Blog, August 2022.
- [7] Zachary DeVito. A guide to PyTorch’s CUDA caching allocator. Zach DeVito’s Blog, August 2022.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.
- [9] Sahib Dhanjal. Mastering GPU memory management with PyTorch and CUDA. Level Up Coding, March 2025.
- [10] Quentin Gallouédec. Visualize and understand GPU memory in PyTorch. Hugging Face Blog, December 2024.
- [11] Audrunas Gruslys, Rémi Munos, Ivo Danihelka, Marc Lanctot, and Alex Graves. Memory-efficient backpropagation through time. *CoRR*, abs/1606.03401, 2016.
- [12] Yueming Hao, Xu Zhao, Bin Bao, David Berard, Will Constable, Adnan Aziz, and Xu Liu. Torchbench: Benchmarking pytorch with high api surface coverage, 2023.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.
- [14] Hey Amit. Mastering memory profiling in PyTorch. Biased-Algorithms, n.d.
- [15] Hugging Face Contributors. Methods and tools for efficient training on a single GPU. Hugging Face Transformers Documentation, 2024.

- [16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [17] Vivek Kini and Jake Hemstad. Using the NVIDIA CUDA stream-ordered memory allocator, part 1. NVIDIA Technical Blog, July 2021.
- [18] Ya Le and Xuan S. Yang. Tiny imagenet visual recognition challenge. 2015.
- [19] Lightning AI. Faster PyTorch training by reducing peak memory. Lightning AI, June 2023.
- [20] Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. *CoRR*, abs/1711.05101, 2017.
- [21] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *CoRR*, abs/1609.07843, 2016.
- [22] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *CoRR*, abs/1710.03740, 2017.
- [23] Szymon Migacz. PyTorch performance tuning guide. PyTorch Tutorials, September 2020.
- [24] NVIDIA Corporation. CUDA runtime api: Stream ordered memory allocator. NVIDIA CUDA Toolkit Documentation, n.d.
- [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. *CoRR*, abs/1912.01703, 2019.
- [26] PyTorch Contributors. Automatic mixed precision examples. PyTorch Documentation, n.d.
- [27] PyTorch Contributors. Automatic mixed precision package: `torch.amp`. PyTorch Documentation, n.d.
- [28] PyTorch Contributors. CUDA semantics: Memory management. PyTorch Documentation, n.d.
- [29] PyTorch Contributors. CUDA semantics: Understanding cuda memory usage. PyTorch Documentation, n.d.

- [30] PyTorch Contributors. Implementing high-performance transformers with scaled dot product attention. PyTorch Tutorials, n.d.
- [31] PyTorch Contributors. PyTorch Memory Snapshot Visualizer. PyTorch Documentation, n.d.
- [32] PyTorch Contributors. `torch.nn.functional.scaled_dot_product_attention`. PyTorch Documentation, n.d.
- [33] PyTorch Contributors. `torch.utils.checkpoint`. PyTorch Documentation, n.d.
- [34] PyTorch Foundation. Current and new activation checkpointing techniques in PyTorch. PyTorch Blog, March 2025.
- [35] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [36] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimization towards training A trillion parameter models. *CoRR*, abs/1910.02054, 2019.
- [37] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. Virtualizing deep neural networks for memory-efficient neural network design. *CoRR*, abs/1602.08124, 2016.
- [38] Aaron Shi and Zachary DeVito. Understanding GPU memory 1: Visualizing all allocations over time. PyTorch Blog, December 2023.
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [40] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *CoRR*, abs/1804.07461, 2018.
- [41] Guanchu Wang, Zirui Liu, Zhimeng Jiang, Ninghao Liu, Na Zou, and Xia Hu. Division: Memory efficient training via dual activation precision, 2023.